

Sentinels — Learning Notes

How the project was built, one concept at a time

August 2, 2026

Contents

1. [How to use these notes](#)
2. [A1 — Environment up](#)
3. [A2 — Server breathes](#)
4. [A3 — The contract](#)
5. [A4 — First real agent](#)
6. [A5 — Orchestrator v1](#)
7. [A6 — Scoring](#)
8. [A7 — Recon agent](#)

9. A8 — TLS agent
10. A9 — Exposure agent
11. A10 — DNS agent
12. A11 — Parallel
13. A12 — AI analyst
14. A13 — Next.js up
15. A14 — It talks
16. A15 — Report view
17. A16 — Live progress and Server-Sent Events
18. A17 — PDF export and Playwright
19. A17b — Scroll drift and refs
20. A17c — Drift speeds and clamping
21. A17d — Findings evidence as artifacts
22. A17e — Screen 2: the set piece
23. 17f — Two event loops on Windows, and the `--reload` trap
24. 18 — Shipping: docs as a contract, and clean-room verification

How to use these notes

One note per achievement (see [../ROADMAP.md](#)). Written *alongside* the code, not after — so if the code exists, the note explaining it exists too.

What's in every note

Each note follows the same five sections:

1. What we built, in one paragraph

Plain English. No jargon. If you read only this, you still know what changed.

2. New concepts

Every idea the code introduces, each with a **tiny standalone example** — 3–10 lines you can paste into a file and run on its own, with no rest-of-the-project needed. The example is always about something ordinary, *not* about security scanning, so the concept stays separable from our project.

3. The actual code, walked through

Our real code, in chunks, with the reasoning for each decision — including the choices we *rejected* and why.

4. Try this

Small experiments: break it on purpose, change a value, see what happens. This is where it actually sticks.

5. Words you now know

A short glossary of the terms introduced, so you can search them later.

A worked sample of the style

Here's the level of explanation to expect. Say the code contained this line:

```
findings: list[Finding] = Field(default_factory=list)
```

The note would explain it like this:

Concept: mutable default arguments

*In Python, a default value is created **once**, when the function or class is first defined — not each time it's used. For immutable values (numbers, strings) that's harmless. For a mutable value like a list, it's a classic bug: every instance ends up sharing the same list.*

Standalone example — run this and watch it go wrong:

```
``python def add_topping(topping, pizza=[]): # the  
bug: one list, shared forever pizza.append(topping)  
return pizza
```

```
print(add_topping("cheese")) # ['cheese'] — fine  
print(add_topping("olives")) # ['cheese', 'olives'] —  
wrong! expected ['olives'] ``
```

The second call was supposed to start fresh, but it inherited the first call's list.

The fix is to build a new list each time:

```
python def add_topping(topping, pizza=None): if pizza
is None: pizza = [] # a brand-new list, every call
pizza.append(topping) return pizza
```

`Field(default_factory=list)` is Pydantic's version of exactly that fix. `list` here is the function — not `list()`, the result of calling it. Pydantic calls it fresh for every new object, so each `AgentResult` gets its own findings list. In our code, this is what stops the Headers agent's findings from leaking into the TLS agent's results.

That's the standard: **the concept, an ordinary example, the mistake it prevents, then where it lands in our code.**

Reading order

Notes are numbered to match achievements, and each assumes the ones before it. Read in order the first time. After that they work fine as reference — jump to whichever one covers what you've forgotten.

If something isn't clear

Say so, and name the specific line. Vagueness in these notes is a bug in them, not a gap in you — I'll rewrite the section rather than re-explain it verbally.

A1 — Environment up

Status: done. `backend/.venv/` exists with 27 packages installed, all imports verified.

1. What we built

We gave the backend its own private Python installation — a *virtual environment* — and installed the 8 libraries listed in `requirements.txt` into it. Nothing was installed into your system Python, so this project can't break your other projects and they can't break it. We also added a `.gitignore` so those installed packages never get committed to git.

No application code yet. This is the foundation everything else stands on.

2. New concepts

Concept: the virtual environment

Python installs packages into one shared folder by default. That's fine until two projects disagree about a version.

Standalone example of the problem — imagine two folders on your machine:

```
project-a/  needs  requests==2.20  (old, written 2019)
project-b/  needs  requests==2.32  (new, written 2026)
```

With one shared Python, installing `requests` for project-b **overwrites** the version project-a depends on. Project-a silently breaks. There is no version of "install both" that works — there's one folder, one `requests`.

A virtual environment is just **a folder containing its own copy of Python and its own `site-packages`**. Each project gets one. They can't see each other.

```
python -m venv .venv          # creates the folder
```

That's genuinely all it is — a directory. Look inside ours:

```
backend/.venv/
  Scripts/python.exe    <- a Python that looks in ITS
OWN site-packages
  Lib/site-packages/    <- our 27 packages live here,
nowhere else
```

The important consequence: **which** `python.exe` **you run** **decides which packages exist.**

```
python -c "import fastapi"                # system
Python -> ModuleNotFoundError
./venv/Scripts/python.exe -c "import fastapi" # venv
Python -> works
```

This is the #1 source of "but I installed it!" confusion. You didn't install it *for that Python*.

Why `.venv` **and not** `venv` ? The leading dot is convention for "tooling, not source code" — editors and file listings sort it out of the way, and VS Code

auto-detects a folder named `.venv` as the project interpreter.

Concept: `python -m pip` instead of `pip`

We ran `./.venv/Scripts/python.exe -m pip install ...`, not `pip install ...`.

`-m` means "run this module as a script, using **this specific** Python."

Standalone example of why it matters:

```
pip install requests           # WHICH pip?
whichever the PATH found first
python -m pip install requests  # unambiguous: THIS
python's pip, this python's packages
```

Remember `where python` earlier returned **three** different Pythons on your machine:

```
C:\Users\Ariha\AppData\Local\Programs\Python\Python313\python.exe
```

```
Y:\python.exe
```

```
C:\Users\Ariha\AppData\Local\Microsoft\WindowsApps\python.exe
```

With three Pythons installed, a bare `pip` is a coin flip.

`python -m pip` never is. Make it a habit.

Concept: pinning versions with `==`

Our `requirements.txt` uses exact versions:

```
fastapi==0.115.6
```

The three options, and what each means:

```
fastapi          # any version – whatever is newest
the day you install

fastapi>=0.115.6  # at least this, but newer is fine

fastapi==0.115.6  # exactly this. nothing else.
```

Standalone example of why unpinned hurts: you build and demo your project in July on `fastapi 0.115`. In September, FastAPI 0.120 ships with a renamed function.

A judge or teammate clones your repo, runs `pip install -r requirements.txt`, gets 0.120, and the app crashes on import — **on code you never touched**. You can't reproduce it, because your machine still has July's version.

Pinning trades "automatic updates" for "it runs the same everywhere, forever." For a project you'll demo, that trade is obviously correct.

Concept: transitive dependencies

You listed **8** packages. Pip installed **27**. The other 19 are dependencies *of your dependencies*.

Standalone example of the shape:

```
you asked for: fastapi
fastapi needs: starlette, pydantic
starlette needs: anyio
anyio needs: sniffio, idna
```

Ask for one thing, get a tree. A few from our install worth recognizing when they show up in error messages later:

Package	Arrived because of	What it does
<code>starlette</code>	<code>fastapi</code>	The actual web machinery — FastAPI is a layer on top
<code>pydantic-core</code>	<code>pydantic</code>	Pydantic's validation engine, written in Rust for speed
<code>anyio</code> / <code>sniffio</code>	<code>fastapi</code> , <code>httpx</code>	Async plumbing shared between libraries
<code>h11</code> / <code>httpcore</code>	<code>httpx</code>	The raw HTTP protocol implementation
<code>certifi</code>	<code>httpx</code>	The list of trusted certificate authorities — we'll meet this again in the TLS agent (A8)
<code>soupsieve</code>	<code>beautifulsoup4</code>	CSS selector support for HTML parsing

You never import these directly. But when a traceback mentions `h11`, now you know it came from `httpx`, not from nowhere.

Concept: why `.venv/` must never be committed

Our `.gitignore` includes `.venv/`. Two reasons:

1. **Size** — those 27 packages are tens of thousands of files. Git would crawl.
2. **They aren't portable** — `.venv` contains compiled binaries built for *your* Windows machine and *your* Python 3.13. On someone's Mac, they're useless.

`requirements.txt` is the portable thing. It's a *recipe*, not the meal:

```
requirements.txt    9 lines, works everywhere    -> COMMIT  
THIS  
  
.venv/              ~30,000 files, Windows-only -> IGNORE  
THIS
```

Anyone can rebuild `.venv` from the recipe in 60 seconds. That's the whole point.

While we're here — the same file also ignores `.env` but **not** `.env.example`:

```
.env                real secrets (your actual API key) ->  
IGNORED, never committed
```

```
.env.example    the template (ANTHROPIC_API_KEY=)    ->
committed, shows what's needed
```

Committing a real `.env` is one of the most common ways API keys leak publicly. The `.gitignore` line is what prevents it.

3. The actual commands, walked through

```
cd backend

python -m venv .venv
```

Creates `backend/.venv/`. Uses your system Python once — the last time we'll do that.

```
./venv/Scripts/python.exe -m pip install --upgrade pip -
-quiet
```

Upgrades pip *inside* the venv first. The bundled pip is often outdated, and an old pip sometimes can't read newer package formats. Cheap insurance.

Windows note: it's `.venv/Scripts/` here. On Mac/Linux it's `.venv/bin/`. If you ever follow a tutorial that says `bin` and it doesn't exist, that's why.

```
./venv/Scripts/python.exe -m pip install -r  
requirements.txt
```

`-r` = "read the list from this file." Pip resolves the whole dependency tree, then downloads and installs all 27.

Then the verification — the part that actually proves A1 is done:

```
import fastapi, httpx, dns.resolver, bs4, anthropic,  
pydantic  
  
from models import Finding, Severity, Status  
f = Finding(id='missing-hsts', title='HSTS header not  
set', category='Headers',  
            severity=Severity.HIGH, status=Status.FAIL)
```

Two things checked at once. The imports prove the packages are installed *and reachable from this Python*. Building a real `Finding` proves `models.py` — written before you and I started — actually works against the installed Pydantic version.

Note `import dns.resolver`, not `import dnspython`. **The install name and the import name are different.** This is common and catches everyone:

```
pip install dnspython      -> import dns
pip install beautifulsoup4 -> import bs4
pip install python-dotenv  -> import dotenv
```

The output confirmed the scoring constant wired up correctly too:

```
models.py works -> HSTS header not set | High | penalty
15
```

That `15` came from `SEVERITY_PENALTY[Severity.HIGH]` in `models.py` — the same deduction table you'll see in the score in A6.

4. Try this

See the isolation for yourself. Run both and compare:

```
python -c "import fastapi; print('found it')"
```

That should **fail** with `ModuleNotFoundError` — your system Python has no fastapi. Then:

```
cd backend && ./venv/Scripts/python.exe -c "import  
fastapi; print('found it')"
```

That works. Same machine, same command, different Python. That contrast *is* the concept.

Count what actually got installed:

```
cd backend && ./venv/Scripts/python.exe -m pip list
```

27 entries from 8 requested. Find `starlette` in the list — you never asked for it; FastAPI did.

See the dependency tree for one package:

```
cd backend && ../venv/Scripts/python.exe -m pip show  
fastapi
```

The `Requires:` line names its direct dependencies. The `Required-by:` line is empty — nothing depends on fastapi, because fastapi is what *we* wanted.

5. Words you now know

- **Virtual environment (venv)** — a folder holding a project's own Python and its own packages, isolated from the system and from other projects.
- `site-packages` — the directory inside a Python installation where third-party packages actually live.
- **Pinning** — specifying an exact dependency version with `==`, so installs are reproducible.
- **Transitive dependency** — a package you didn't ask for, installed because something you did ask for needs it.

- **-m flag** — "run this module using *this* Python," removing ambiguity about which Python or which pip you meant.
 - **.gitignore** — a list of paths git should never track; here it protects us from committing both bulk (`.venv/`) and secrets (`.env`).
-

Next: [A2 — Server breathes](#). We write the first real code: a FastAPI app with a `/health` endpoint you can open in a browser.

A2 — Server breathes

***Status:** done. `backend/main.py` serves `/health` and `/` on port 8000.*

What we built

A file, `backend/main.py`, that answers when you visit `localhost:8000/health` in a browser. That's it. No scanning yet — just a server that responds.

The one big idea

When your browser asks for `/health`, the request passes through **three things**, in order:

Browser → Uvicorn → FastAPI → your function

- **Browser** — sends plain text over the network: "give me `/health`".

- **Uvicorn** — a program that owns the actual network connection. It reads that text and hands it to FastAPI.
- **FastAPI** — doesn't touch the network at all. It just keeps a list of "when this URL is asked for, call this function" and looks yours up.
- **Your function** (`health()`) — plain Python. It has no idea a browser or the internet exists. It just returns a dict.

That's the whole architecture. Everything else in this note is detail on top of that one picture.

How you tell FastAPI "call this function for this URL"

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

That `@app.get("/health")` line is called a **decorator**. All it does is add one entry to FastAPI's lookup list: *"when someone asks for GET /health, call this function."* It

doesn't change your function at all — `health()` still works exactly like a normal function if you call it directly yourself.

Think of it like writing a name on a sign-up sheet. Writing your name down doesn't change who you are — it just means someone with the sheet in hand knows to call you when needed.

The actual file

```
from fastapi import FastAPI

app = FastAPI(title="Sentinels")

@app.get("/")
def root():
    return {"service": "Sentinels", "health": "/health"}

@app.get("/health")
def health():
    return {"status": "ok", "service": "Sentinels"}
```

- `app = FastAPI(...)` creates that lookup list.
- `/` is just a friendly landing message, so visiting the bare address doesn't look broken.
- `/health` is the one that actually answers "is the server alive?" It deliberately does nothing fancy — no database check, no network call — because a health check that can fail for unrelated reasons is a bad health check.

Try it

Start the server:

```
cd backend && ./venv/Scripts/python.exe -m uvicorn  
main:app --reload
```

Then:

- Open `localhost:8000/health` — you should see `{"status": "ok", ...}`.
- Open `localhost:8000/docs` — a free, auto-generated page listing every route, built by FastAPI just from reading your code.

- Try a URL that doesn't exist, e.g. `localhost:8000/nope` — you get a `404`. That's the lookup list being asked for something it doesn't have.

A couple of words worth knowing

- **Uvicorn** — the program that actually owns the network connection. FastAPI never touches a socket directly.
 - **Decorator** (`@something`) — a way of registering a function without changing it.
 - `--reload` — restarts the server automatically whenever you save a file, so you don't have to stop and start it by hand while developing.
 - `main:app` — tells uvicorn "open `main.py`, and use the variable called `app` inside it."
-

Next: A3 — The contract. We write the shared shape every scanning agent will follow.

A3 — The contract

Status: done. `backend/agents/base.py` defines `ScanContext` and `BaseAgent`. **Verified:** a bare `BaseAgent()` refuses to instantiate, a working subclass returns a populated `AgentResult`, and a subclass whose `scan()` raises still returns a clean `AgentResult` with the error captured instead of crashing.

What we built

Five agents (Headers, Recon, TLS, Exposure, DNS) are coming. They all need to look the same from the outside — same inputs, same output — or the orchestrator that calls them would need a special case for each one.

`backend/agents/base.py` is that shared shape:

- **ScanContext** — a small bundle of "everything an agent needs" (the target URL and a shared HTTP connection), built once per scan and handed to all five agents.

- `BaseAgent` — a base class that *requires* every subclass to write a `scan()` method, and in return gives it a `run()` method for free that adds timing and crash-proofing.

No real scanning yet — this is just the socket every future agent plugs into.

The one big idea: `async` and `await`

Normal Python runs one line, waits, runs the next. That's fine until a line means "wait for the network" — that can take hundreds of milliseconds, during which a normal program just sits frozen. If we're running five agents, we don't want four of them stuck waiting for the fifth to finish its network call before they even start theirs.

`async` / `await` lets Python pause a function at the exact point it's waiting on something, and go do other work in the meantime. Standalone example, nothing to do with security — boiling two eggs:

```
import asyncio
```

```
async def boil_egg(name, seconds):  
    print(f"{name}: water on")  
    await asyncio.sleep(seconds)    # "wait", without  
freezing everything else  
    print(f"{name}: done")  
  
async def main():  
    await asyncio.gather(  
        boil_egg("egg A", 2),  
        boil_egg("egg B", 2),  
    )  
  
asyncio.run(main())
```

This takes about **2 seconds total, not 4** — both eggs "boil" during the same window. There's still only one thread doing one thing at a time; what's really happening is that while egg A is waiting, Python lets egg B start, and hops back and forth between them at each `await`. `asyncio.gather` is what says "start all of these, and let them take turns waiting" — that's the entire trick behind five

agents finishing in about the time of the slowest one, not the sum of all five.

`scan()` is declared `async def` because every real agent's version of it will `await` a real HTTP call (starting next note). This file has no network code yet, so nothing here proves the speedup — that comes later once there are real requests to time.

The other new idea: abstract base classes

We want Python itself to enforce "every agent must have a `scan()` method" — not a comment we hope people read. Standalone example, nothing about security:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

class Square(Shape):
    def __init__(self, side):
```

```

        self.side = side

    def area(self) -> float:
        return self.side ** 2

Shape()          # TypeError: can't instantiate abstract
class Shape
Square(4).area() # 16 – fine

```

If a subclass forgets to write `area()`, trying to create it raises a `TypeError` immediately — not later, with a confusing error from whoever eventually calls `.area()`.

`BaseAgent` uses the same pattern: `scan()` is `@abstractmethod`, so a subclass that forgets it can't even be constructed.

The actual code

```

@dataclass
class ScanContext:
    url: str                # normalized target, e.g.
    "https://example.com"

```

```
client: httpx.AsyncClient    # one shared connection
pool for all 5 agents
```

`@dataclass` just auto-generates `__init__` from the two fields above — no need to write it by hand. `ScanContext` is built once per scan and passed to every agent, so no agent opens its own connection or re-figures-out the target URL.

```
class BaseAgent(ABC):
    name: str = "base"

    @abstractmethod
    async def scan(self, context: ScanContext) ->
list[Finding]:
    raise NotImplementedError

    async def run(self, context: ScanContext) ->
AgentResult:
    start = time.perf_counter()
    error: str | None = None
    findings: list[Finding] = []
    try:
```

```

        findings = await self.scan(context)

    except Exception as exc:

        error = f"{type(exc).__name__}: {exc}"

        duration_ms = int((time.perf_counter() - start) *
1000)

        return AgentResult(

            agent=self.name, findings=findings,

            duration_ms=duration_ms, error=error,

        )

```

- Subclasses only ever write `scan()`. They inherit `run()` unchanged — so timing and error-catching happen exactly the same way for every agent, instead of five people each having to remember to write their own `try/except`.
- `except Exception` is deliberately broad. That's the whole point: whatever goes wrong inside one agent (a DNS error, a timeout, a bug we didn't anticipate), it must never take down the other four agents' scans. That's the CLAUDE.md rule: agents never crash the scan.
- `findings` starts as `[]` before the `try`, so even if `scan()` raises immediately, `run()` still has a valid (empty) list

to return.

Try it

Write a subclass that deliberately raises inside `scan()`, call `.run()` on it, and check you get a clean `AgentResult` back with `error` set and `findings=[]` — no traceback, no crash.

Then write a subclass that forgets to define `scan()` at all and try to instantiate it — confirm you get `TypeError:`

```
Can't instantiate abstract class ... without an  
implementation for abstract method 'scan'.
```

Paste the `boil_egg` example above into a file and run it, then change `asyncio.gather(...)` to `await boil_egg(...)` twice in a row instead — watch the total time roughly double.

Words worth knowing

- **Coroutine** — a function defined with `async def`; calling it doesn't run it, it hands back a paused "plan" that something else has to drive.

- `await` — the only point a coroutine can pause, so other work can run during a wait.
 - **Abstract base class (ABC)** — a class that can't be instantiated until every `@abstractmethod` has been overridden by a subclass.
 - **Dependency passing** — building a shared resource (like the HTTP client) once, from the outside, and handing it in, instead of letting every agent build its own.
-

Next: A4 — First real agent. `HeadersAgent` becomes the first subclass of `BaseAgent` that does something real: an actual HTTP request against a live site, checking for missing security headers.

A4 — First real agent

Status: done. `backend/agents/headers.py` makes one real, live HTTP GET request and checks four security-relevant response headers. Verified against `https://example.com` (fails all four — it sets none of them) and `https://github.com` (passes all four, with real header values captured as evidence).

What we built

`HeadersAgent`, the first subclass of `BaseAgent` (from A3) that actually does something. It makes one HTTP GET request to the target URL, checks four response headers that browsers use to enforce security rules, and produces one `Finding` per header — `FAIL` if it's missing, `PASS` if it's present, with the real header value kept as evidence either way.

The four headers: `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options`, `X-Frame-Options`. All four are read-only observations from a normal

page load — nothing here sends anything a regular visitor's browser wouldn't already send.

This is also the first code in the project that talks to the real internet, and the first real test of A3's `run()` / `scan()` contract: if the request fails (DNS error, timeout, connection refused), `scan()` raises and A3's `run()` catches it automatically — nothing new had to be written for that.

The one big idea: HTTP response headers

When a server answers a request, it doesn't just send back a page — it sends a block of `Name: value` lines first, called headers. They carry metadata: how big the response is, what type of content it is, and — what this project cares about — instructions to the browser about what security rules to enforce.

Try it yourself:

```
curl -I https://example.com
```

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html; charset=UTF-8
```

```
Content-Length: 1256
```

A security header is just one more line in that same block

— `Strict-Transport-Security: max-age=31536000` is

structurally no different from `Content-Type: text/html`.

The server doesn't do anything special to send one; what makes it a "security" header is what the **browser** promises to do when it sees that name (refuse to load in a frame, refuse to downgrade to HTTP, etc). It's an instruction to the browser, not something the server enforces itself.

Making the actual request

```
response = await context.client.get(context.url,  
follow_redirects=True)  
headers = response.headers
```

`context.client` is the shared `httpx.AsyncClient` from `ScanContext` (A3) — the thing that actually speaks HTTP.

`await client.get(...)` is a real network wait: exactly the kind of pause A3's egg example was standing in for. While this one request is in flight, the event loop is free to run the other four agents' requests at the same time.

Two details worth knowing:

- `follow_redirects=True` — by default `httpx` does *not* follow redirects; a GET to a URL that responds `301 Moved Permanently` just hands back that redirect, not the page it points to. Redirect responses rarely set meaningful security headers, so without this we'd sometimes grade the wrong page.
- `response.headers` **is case-insensitive**. HTTP header names officially don't care about case, and different servers really do send them differently (`X-Frame-Options` VS `x-frame-options`). `headers.get("x-frame-options")` works no matter how the server capitalized it — a plain Python `dict` would not do this for you.

The actual code

```

async def scan(self, context: ScanContext) ->
list[Finding]:
    response = await context.client.get(context.url,
follow_redirects=True)
    headers = response.headers
    return [
        self._check(headers, "content-security-policy",
id_"missing-csp", ...),
        self._check(headers, "strict-transport-security",
id_"missing-hsts", ...),
        self._check(headers, "x-content-type-options",
...),
        self._check(headers, "x-frame-options", ...),
    ]

```

One request, four checks against the same response — faster than four separate requests, and it matches what one real page load actually looks like.

```

@staticmethod
def _check(headers, header_name, *, id_, title, severity,
description="", remediation=""):

```

```

value = headers.get(header_name)

if value is None:

    return Finding(..., status=Status.FAIL,
severity=severity,

                                evidence=f"No '{header_name}'
header in the response.")

    return Finding(..., status=Status.PASS,
severity=Severity.INFO,

                                evidence=f"{header_name}: {value}")

```

Worth noticing:

- We return a `Finding` for **PASS as well as FAIL**. If we only reported problems, the report couldn't tell "we checked this and it was fine" from "we never checked this at all" — and a report that's a wall of failures with nothing passing also just looks broken.
- The `*` before `id_` makes every argument after it keyword-only — you must call `_check(headers, name, id_="x", title="y", ...)`, not positionally. `_check` takes seven arguments; without this, swapping `title` and `id_` by accident wouldn't error, it would just quietly produce a finding with the wrong label.

- `@staticmethod` because `_check` doesn't touch `self` — it's a plain function that happens to live on the class.

Try it

Run `HeadersAgent` against `https://example.com` (fails everything) and then `https://github.com` (passes everything) and compare the findings.

Point it at a URL that can't resolve at all, e.g. `https://this-domain-does-not-exist-zzzz.invalid`, and confirm you get a clean `AgentResult` back with `error` set and `findings=[]` — not a crash. That's A3's `run()` catching a *real* failure for the first time.

Compare the raw headers with `curl -sI https://github.com` against what the agent reports as evidence — they should match exactly.

Words worth knowing

- **HTTP response header** — a `Name: value` metadata line sent before a response body; some names are instructions the browser is expected to enforce.

- `httpx.AsyncClient` — the async HTTP client this project uses; one shared instance per scan, passed in via `ScanContext` .
 - `follow_redirects` — an `httpx` option; without it, a redirecting URL hands back the redirect itself, not the page it points to.
 - **Keyword-only arguments** (`*` in a signature) — arguments after it can only be passed by name, turning an argument-order mistake into an immediate error.
-

Next: A5 — Orchestrator v1. A real `/scan` POST endpoint that normalizes a URL, runs `HeadersAgent` , and returns a real `ScanReport` — the first end-to-end request/response cycle in the project.

A5 — Orchestrator v1

Status: *done.* `POST /scan` normalizes a `URL`, runs `HeadersAgent` against it, and returns a real `ScanReport`. Verified against the golden path, an empty `URL`, a missing field, an unsupported scheme, and an unresolvable domain — five different failure shapes, none of them a crash.

What we built

Three small pieces that together make the first true end-to-end request:

- `ScanRequest` (`models.py`) — the shape of the JSON body a client sends: `{"url": "..."} .`
- `normalize_url()` and `run_scan()` (`orchestrator.py`) — turn whatever string the user typed into one canonical `URL`, run every registered agent against it (for now just `HeadersAgent`), and assemble a `ScanReport` .

- `POST /scan` (`main.py`) — the endpoint. Accepts a `ScanRequest`, calls `run_scan()`, returns a `ScanReport`, and turns a bad URL into a clean `400` instead of a crash.

```
curl -X POST localhost:8000/scan -d
```

```
'{"url": "example.com"}'
```

 now returns a real report with real findings inside it.

The one big idea: URL normalization

People type URLs inconsistently — `example.com`, `EXAMPLE.com/`, `https://example.com` are all "the same site" to a human but three different strings to a computer. If we scanned them as different targets, the same site could get scanned differently just because of how someone capitalized it.

```
from urllib.parse import urlsplit

parsed = urlsplit("https://Example.COM/Path?x=1")
print(parsed.scheme)    # 'https'
```

```
print(parsed.netloc)    # 'Example.COM'
print(parsed.path)      # '/Path'
```

`urlsplit` breaks a URL into its named pieces instead of us hand-writing regexes to find where the host ends and the path begins. `normalize_url()` lowercases the scheme and host (both are case-insensitive by spec) but leaves the path alone (`/Path` and `/path` really can be different pages on a real server).

The request lifecycle, and why some failures are 400 and some are 422

Different kinds of bad input get caught at different stages, and that's exactly what decides the HTTP status code:

- `{}` (no `url` key at all) — caught by Pydantic before our code even runs. That's a **422**: the request isn't even shaped like a valid request.
- `{"url": " "}` or `{"url": "ftp://x.com"}` — shaped correctly, but `normalize_url()` rejects the value itself (empty string, unsupported scheme). That's a **400**: our own code checked it and said no.

- **A URL that can't resolve** (bad DNS) — this isn't even an error at the endpoint level. `HeadersAgent`'s request fails, A3's `run()` catches it, and the scan still returns a normal `200` with the failure recorded inside `agents[0].error`.

```
try:
    return await run_scan(request.url)
except ValueError as exc:
    raise HTTPException(status_code=400, detail=str(exc))
from exc
```

`HTTPException` is FastAPI's way of saying "stop, and send back this exact status code and message" instead of a generic crash. `from exc` keeps the original error attached underneath in the traceback, so a 400 months from now still shows *why* — it doesn't just say "rejected," it shows the real `ValueError` message that caused it.

A real bug, briefly

The first version of the scheme check only matched `http://` or `https://` at the start of a URL, to decide "does

this already have a scheme?" That meant

`ftp://example.com` looked like it had *no* scheme, so the code prepended `https://` — producing the nonsense URL `https://ftp://example.com`, which then failed with a confusing DNS error several steps later instead of a clean rejection. The fix widens the check to recognize *any* `word://` prefix, so an unsupported scheme like `ftp` is correctly detected and rejected with a clear `400` instead of silently mangled.

The actual code

```
async def run_scan(raw_url: str) -> ScanReport:
    url = normalize_url(raw_url)
    start = time.perf_counter()

    async with httpx.AsyncClient(timeout=10.0) as client:
        context = ScanContext(url=url, client=client)
        agent_results = [await agent_cls().run(context)
                          for agent_cls in AGENTS]

    findings = [finding for result in agent_results for
```

```
finding in result.findings]
```

```
...
```

- `timeout=10.0` matters here for real: without it, `httpx` waits indefinitely for a hung server, and one slow site could stall a scan forever.
- The last line flattens a list of lists (each agent's own `findings`) into one flat list for the report — read it as nested loops: for each agent's result, for each finding in it, collect it.

```
@app.post("/scan", response_model=ScanReport)
async def scan(request: ScanRequest) -> ScanReport:
    try:
        return await run_scan(request.url)
    except ValueError as exc:
        raise HTTPException(status_code=400,
                             detail=str(exc)) from exc
```

`response_model=ScanReport` does two things: validates whatever `run_scan()` returns against the `ScanReport` shape before sending it (so a bug that returns the wrong thing is

caught immediately, not shipped to a client), and documents the exact response shape in `/docs`.

This is also the first endpoint that genuinely needs `async def` — unlike `/health`, `/scan` actually awaits something (`run_scan`, which awaits a real HTTP request inside `HeadersAgent`). A plain `def` endpoint can't contain an `await` at all.

Try it

Start the server and run the golden path:

```
curl -X POST localhost:8000/scan -H "Content-Type: application/json" -d '{"url": "example.com"}'
```

Notice the response's `"url"` comes back as `"https://example.com/"` even though you typed no scheme.

Try each of these and check the status code: `{}` (expect 422), `{"url": " "}` (expect 400), `{"url": "ftp://x.com"}` (expect 400 with a clear message), and an unresolvable

domain (expect 200, with the error inside

```
agents[0].error ).
```

Words worth knowing

- **URL normalization** — reducing equivalent-but-differently-typed URLs to one canonical string.
- **422 vs 400** — 422 means the request body isn't even shaped right (caught automatically before our code runs); 400 means it's shaped right but the value inside is invalid by our own rules.
- **HTTPException** — FastAPI's way of stopping a request with a specific status code and message instead of a generic crash.
- **response_model** — tells FastAPI to validate the returned object's shape and document it in the API docs.

Next: A6 — Scoring. A pure function that turns a list of findings into a 0–100 score and an A–F grade, deterministically.

A6 — Scoring

Status: *done.* `backend/scoring.py` turns a list of findings into a 0-100 score, an A-F grade, and a per-severity count.

What we built

`backend/scoring.py` — three small functions:

- `calculate_score(findings)` — starts at 100, subtracts a penalty for every finding that isn't a clean pass, and never goes below 0.
- `grade_for_score(score)` — turns that number into a letter grade using a fixed table of cutoffs (90+ is A, 80+ is B, and so on).
- `count_by_severity(findings)` — how many non-passing findings fall into each severity bucket, so the report can say "2 High, 2 Medium."

`orchestrator.py` calls all three and puts the real values into the report.

The one big idea: pure functions

A function is **pure** if, given the same input, it always gives the same output — and it doesn't change anything else while it runs (no printing, no saving to a file, no checking the clock).

```
import random

def add(a, b):
    return a + b                    # PURE: same a, b ->
    always same result

def add_with_bonus(a, b):
    return a + b + random.randint(0, 5)  # NOT pure:
    different result every call
```

`calculate_score` is written like `add`. That's exactly what CLAUDE.md means by "scoring stays deterministic — same site, same score, always." It's not just a promise — `scoring.py` doesn't even import anything that *could* break it (no `httpx`, no `datetime.now()`, no AI calls). This also

means you can test it with a plain list of fake findings, no server or internet needed — see "Try it" below.

The actual code

```
def calculate_score(findings: list[Finding]) -> int:
    penalty = sum(
        SEVERITY_PENALTY[finding.severity]
        for finding in findings
        if finding.status != Status.PASS    # FAIL and
WARN both cost something
    )
    return max(0, 100 - penalty)    # never go below 0
```

`max(0, ...)` is called "clamping" — it stops the score from going negative. Five failed `Critical` checks would otherwise do `100 - 125 = -25`, which is meaningless to show a user. `max()` just picks the bigger of the two numbers, so once the subtraction goes below zero, `0` wins instead.

```
_GRADE_THRESHOLDS = [(90, "A"), (80, "B"), (70, "C"),  
(60, "D")]
```

```
def grade_for_score(score: int) -> str:  
    for threshold, grade in _GRADE_THRESHOLDS:  
        if score >= threshold:  
            return grade  
    return "F"
```

Checked **highest first**, on purpose — if the list were reversed, a score of 95 would match `>= 60` first and wrongly come back "D".

```
def count_by_severity(findings: list[Finding]) ->  
dict[str, int]:  
    counts = {severity.value: 0 for severity in Severity}  
    # every bucket starts at 0  
    for finding in findings:  
        if finding.status != Status.PASS:  
            counts[finding.severity.value] += 1  
    return counts
```

That `{severity.value: 0 for severity in Severity}` line is a **dict comprehension** — it pre-fills every severity (Critical, High, Medium, Low, Info) at 0 before counting anything. Without it, a perfectly clean site would return `{}`, and the frontend couldn't tell "everything passed" apart from "nothing was checked."

Try it

Run scoring with zero network involved — the actual point of it being pure:

```
cd backend

./venv/Scripts/python.exe -c "
from models import Finding, Severity, Status
from scoring import calculate_score, grade_for_score,
count_by_severity
findings = [Finding(id='a', title='t', category='Test',
severity=Severity.HIGH, status=Status.FAIL)]
score = calculate_score(findings)
print(score, grade_for_score(score),
```

```
count_by_severity(findings))  
"
```

- Build ten fabricated `Critical` / `FAIL` findings and confirm the score clamps at `0`.
- Temporarily reverse `_GRADE_THRESHOLDS` and call `grade_for_score(95)` — wrong grade comes back, proving the ordering matters. Put it back after.
- Scan the same site twice via `/scan` and confirm the score is identical both times.

A couple of words worth knowing

- **Pure function** — same input always gives the same output, no side effects.
 - **Deterministic** — one correct output per input, decided by fixed rules.
 - **Clamping** — forcing a number to stay in a valid range (`max(0, ...)`).
 - **Dict comprehension** — `{key: value for x in iterable}`, builds a dict in one line.
-

Next: Act 2 begins — A7, the Recon agent. This is where HTML parsing (`BeautifulSoup`) enters the project for the first time.

A7 — Recon agent

Status: *done.* `backend/agents/recon.py` — `ReconAgent`, the second real agent, now runs alongside `HeadersAgent` in every scan.

What we built

`ReconAgent` runs two passive checks:

- **Generator meta tag** — parses the homepage's HTML looking for `<meta name="generator" content="...">`, a tag many CMSes (WordPress, Drupal, Wix) add automatically that announces exactly what software and version built the page.
- `robots.txt` — fetches `/robots.txt` and checks its disallowed paths against a short list of sensitive-sounding keywords (`admin`, `login`, `config`, `.env`, etc.).

Both are just a single GET to a public path — nothing here guesses or brute-forces anything. Tested against real sites:

`wordpress.org` reveals its generator tag, `github.com`'s

`robots.txt` hints at a sensitive path, `example.com` has neither.

The one big idea: parsing HTML with BeautifulSoup

A web page's HTML is just text. You *could* search it with plain string matching, but that breaks the moment whitespace or attribute order is slightly different.

BeautifulSoup parses the text into a tree of tags you can query properly, the same way a browser understands the page's structure.

```
from bs4 import BeautifulSoup

html = '<html><body><meta name="author" content="Sam">
</body></html>'

soup = BeautifulSoup(html, "html.parser")

tag = soup.find("meta", attrs={"name": "author"})
print(tag.get("content"))    # 'Sam'
```

- `"html.parser"` tells BeautifulSoup which engine to use — it's the one built into Python, so no extra install is needed.
- `.find(...)` returns the first match (or `None`);
`.find_all(...)` returns every match as a list. Our generator check uses `.find` since a page only ever has one.

Our actual check:

```
soup = BeautifulSoup(response.text, "html.parser")
tag = soup.find("meta", attrs={"name": "generator"})
content = tag.get("content") if tag else None
```

Using `.get("content")` instead of `tag["content"]` matters: if the tag exists but is missing that attribute, `.get(...)` quietly returns `None` instead of crashing.

robots.txt, briefly

`robots.txt` is a plain-text file telling web crawlers which paths they're asked not to index — a polite request, not a lock. The reason a security auditor checks it: a human

wrote that file, and humans tend to list the paths they consider most sensitive, which is a free hint about where something interesting probably lives.

```
User-agent: *  
Disallow: /admin  
Disallow:
```

A bare `Disallow:` with nothing after it means "nothing else is disallowed" — our code filters those empty entries out. One real wrinkle found while testing against `github.com`: its `robots.txt` repeats the same path under several different `User-agent:` blocks, so the check printed it four times over until a one-line fix — `list(dict.fromkeys(disallowed))` — deduped it while keeping the original order.

The actual code

```
async def scan(self, context: ScanContext) ->  
list[Finding]:  
    return [
```

```
        await self._check_generator(context),
        await self._check_robots_txt(context),
    ]
```

Two independent checks against two different URLs — the homepage, and `/robots.txt`. To always hit the real root of the domain (even if the scanned URL was some deep page), we build the robots.txt URL with `urljoin`:

```
from urllib.parse import urljoin
robots_url = urljoin(context.url, "/robots.txt")
# urljoin("https://example.com/blog/post", "/robots.txt")
# -> "https://example.com/robots.txt" (leading "/"
# means "from the root")
```

Plain string concatenation would have produced a broken URL like `https://example.com/blog/post/robots.txt` instead.

A missing `robots.txt` (usually a 404) is reported as `PASS`, not a problem — plenty of well-run sites just don't have one.

Try it

```

cd backend

./venv/Scripts/python.exe -c "
import asyncio, httpx
from agents.base import ScanContext
from agents.recon import ReconAgent

async def main():
    async with httpx.AsyncClient(timeout=10.0) as client:
        for url in ['https://example.com',
'https://github.com', 'https://wordpress.org']:
            ctx = ScanContext(url=url, client=client)
            result = await ReconAgent().run(ctx)
            print(url, [(f.status.value, f.title) for f
in result.findings])

asyncio.run(main())
"

```

- Notice only `wordpress.org` trips the generator-tag check.
- Look at a real `robots.txt` yourself first: `curl -s https://github.com/robots.txt | grep -i disallow.`

- With the server running, POST to `/scan` with `wordpress.org` and check the response now has two agents (`headers` and `recon`) combined into one report.

A couple of words worth knowing

- **BeautifulSoup** — parses HTML text into a queryable tree instead of raw strings.
- `.find()` vs `.find_all()` — first match only, vs. every match as a list.
- `robots.txt` — a request to crawlers, not an enforcement mechanism; visible to anyone.
- `urljoin` — resolves a path against a base URL the way a browser does.
- `dict.fromkeys(items)` — dedupes a list while keeping the original order.

Next: A8 — TLS agent. Reading a real certificate's expiry date using raw sockets and Python's `ssl` module.

A8 — TLS agent

Status: done. `backend/agents/tls.py` — `TLSEAgent`, the third real agent, checks a site's TLS certificate and protocol version.

What we built

`TLSEAgent` is the first agent that doesn't use HTTP at all. Instead of `httpx`, it opens a raw TCP socket and does a real TLS handshake itself, using Python's built-in `ssl` module — the same layer HTTPS has been quietly running on top of the whole time.

Three checks:

- **Is this site even using HTTPS?** If the URL is plain `http://`, that's an instant `Critical` finding — there's no certificate to check at all.
- **Certificate verification** — does a real handshake succeed (trusted chain, correct hostname)? If not, one

finding explains why: expired, self-signed, or wrong hostname.

- **Expiry and protocol version** — if the handshake succeeds, read the real expiry date and the negotiated TLS version off the certificate.

Verified against `badssl.com` (a public test site with deliberately broken certificates) covering expired, self-signed, and hostname-mismatch cases, plus real sites and a plain-HTTP target.

The one big idea: blocking code would freeze the event loop

Everything before this agent used `httpx`, which was built to cooperate with `asyncio` — `await client.get(...)` pauses politely and lets other agents run while it waits. `socket` and `ssl`, by contrast, are old-school **blocking** APIs: once you call `sock.recv()` or `wrap_socket()`, nothing else in the program runs until it finishes.

If we called them directly inside `async def scan()`, the entire server would freeze for the whole handshake —

every other agent's `await` would just sit there stuck behind it.

`asyncio.to_thread` fixes this: it runs an ordinary blocking function on a separate background thread, and hands back something you *can* `await`. From the event loop's point of view it behaves like any other well-behaved async call, even though the function itself has no idea `asyncio` exists.

```
import asyncio, time

def blocking_nap(seconds):
    time.sleep(seconds)      # a REAL block, not
    asyncio.sleep
    return f"slept {seconds}s"

async def ticker():
    for i in range(10):
        print("tick", i)
        await asyncio.sleep(0.1)

async def main():
```

```
# run both AT THE SAME TIME

await asyncio.gather(asyncio.to_thread(blocking_nap,
1), ticker())

asyncio.run(main())
```

The ticks keep printing steadily during that whole second — proof the blocking `time.sleep` didn't freeze anything else. This is exactly the mechanism `TLSAgent` relies on to do a real ~400ms handshake without stalling the rest of the scan.

The actual code

```
def _fetch_certificate(hostname, port, timeout):
    context = ssl.create_default_context()
    with socket.create_connection((hostname, port),
    timeout=timeout) as sock:
        with context.wrap_socket(sock,
    server_hostname=hostname) as ssock:
            return ssock.getpeercert(), ssock.version()
```

A plain, ordinary function — no `async` / `await` inside it at all. That's on purpose: it's the function we hand to `to_thread`, which expects a normal blocking callable to run on another thread. `ssl.create_default_context()` checks the certificate against trusted CAs and the hostname, just like a browser would. If that check fails, this function never returns — it raises `ssl.SSLError` instead.

```
try:
    cert, protocol_version = await
    asyncio.to_thread(_fetch_certificate, hostname, port,
    10.0)
except ssl.SSLError as exc:
    return [Finding(id="tls-cert-invalid", ...,
    evidence=str(exc), ...)]
```

Notice the arguments come *after* the function name — `to_thread(func, arg1, arg2)` — not `to_thread(func(arg1, arg2))`, which would call the function immediately on the main thread and defeat the whole point.

Only `ssl.SSLError` is caught here, deliberately narrow. A DNS failure (unresolvable hostname) raises a completely different exception, `socket.gaierror`, which is left to propagate up to the agent's outer `run()` wrapper (from A3) instead — a DNS failure isn't a fact about this site's TLS setup, it's a fact about whether the site could be reached at all.

```
not_after = datetime.strptime(cert["notAfter"], "%b %d
%H:%M:%S %Y %Z")
```

`cert["notAfter"]` comes back as a fixed-format string like `'Aug 29 21:41:26 2026 GMT'`. `strptime` reads a string according to a template and turns it into a real `datetime` — each `%`-code names one piece (`%b` = abbreviated month, `%Y` = year, and so on).

One thing worth knowing: a certificate's days-until-expiry genuinely changes every day, so this finding can flip from `PASS` to `WARN` tomorrow with zero code changes. That's not a bug — A6's "same input, same score" rule is about the *scoring function*, not about the world staying frozen. The facts change; the rule connecting facts to a score doesn't.

Try it

```
cd backend

./venv/Scripts/python.exe -c "
import asyncio, httpx
from agents.base import ScanContext
from agents.tls import TLSAgent

async def main():
    async with httpx.AsyncClient(timeout=10.0) as client:
        for url in ['https://example.com',
                    'https://expired.badssl.com',
                    'https://self-signed.badssl.com',
                    'http://example.com']:
            ctx = ScanContext(url=url, client=client)
            result = await TLSAgent().run(ctx)
            for f in result.findings:
                print(url, '->', f.status.value, f.title)

asyncio.run(main())
"
```

- `badssl.com` hosts deliberately broken certificates purpose-built for testing tools like this.
- Paste the `blocking_nap / ticker` example above into a file, run it, then remove `to_thread` (bare `time.sleep(1)` inside the coroutine) and watch the ticks stop appearing during that second — that contrast is the whole concept.

A couple of words worth knowing

- **Socket** — an OS handle for one end of a network connection, identified by `(host, port)`.
 - **TLS handshake** — the negotiation where a server presents a certificate before any data flows.
 - `ssl.SSLError` — raised when certificate verification fails; the message names the real reason.
 - `asyncio.to_thread` — runs a blocking function on a background thread so it doesn't freeze the event loop.
 - `strptime` — parses a string into a `datetime` using a format template.
-

Next: A9 — Exposure agent. Detecting a publicly exposed `/.env` file, safely.

A9 — Exposure agent

Status: *done.* `backend/agents/exposure.py` —
`ExposureAgent` checks whether `/.env` or `/.git/HEAD`
are publicly reachable.

What we built

`ExposureAgent` checks two specific paths that should never be reachable on a properly configured server:

- `/.env` — where frameworks keep local secrets (passwords, API keys). If it's sitting in the folder the web server serves from, anyone can just request it.
- `/.git/HEAD` — the front door of an exposed `.git` folder. If reachable, tools exist that rebuild a project's entire history from it.

Each is a single GET request. No guessing, no trying variations — just these two well-known paths.

The one big idea: a 200 doesn't always mean "found it"

A lot of real sites return `200 OK` for *every* URL, even ones that don't exist — single-page apps and custom "not found" pages do this constantly. If the check were just "did `/.env` return 200? Then it's exposed!", it would wrongly flag tons of perfectly safe sites.

So the agent also checks whether the response actually *looks like* the thing it's supposed to be:

```
looks_real = (  
    response.status_code == 200  
    and "html" not in content_type  
    and _ENV_LINE_RE.search(response.text)  # KEY=VALUE  
    lines  
)
```

A real `.env` file is plain-text `KEY=VALUE` lines, not an HTML page. A real `.git/HEAD` file's content always starts with `ref: .` A fake "200 for everything" page matches neither shape, so it correctly gets marked safe. Status code

narrows things down; content shape is what actually confirms it.

Where the ethical line sits

CLAUDE.md says this project only does passive checks — reading things, never attacking. For this agent that means: exactly two fixed, well-known paths, one GET request each, and nothing that tries variations (`.env.bak` , `.env.local` , a wordlist of 500 filenames). One GET to a standard path is what any browser does. Hundreds of GETs guessing at hidden files is fuzzing — which this project has ruled out entirely.

There's also a smaller rule buried in the code: if a `.env` really is exposed, the finding never prints its actual content. It just says how many `KEY=VALUE` lines were found. Otherwise our own report would become a second copy of the leaked secret.

The actual code

```

async def _check_env_file(self, context: ScanContext) ->
Finding:
    env_url = urljoin(context.url, "/.env")
    # follow_redirects=False: if the server redirects
this path away,
    # that's the server telling us "not here" just as
clearly as a 404.
    response = await context.client.get(env_url,
follow_redirects=False)

    looks_real = (
        response.status_code == 200
        and "html" not in content_type
        and _ENV_LINE_RE.search(response.text)
    )

```

`_ENV_LINE_RE` is `re.compile(r"^[A-Za-z_][A-Za-z0-9_]*\s*=", re.MULTILINE)`. The `re.MULTILINE` flag matters: without it, `^` only matches the very start of the whole string. With it, `^` matches the start of *every line*, which is needed since a real `.env` has many `KEY=VALUE` lines and any of them should count.

The `/.git/HEAD` check works the same way, just checking `response.text.strip().startswith("ref:")` instead of the regex.

Try it

Run a tiny server that returns 200 for literally everything (a fake "soft-404" site):

```
from http.server import BaseHTTPRequestHandler,
HTTPServer

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "text/html")
        self.end_headers()
        self.wfile.write(b"<html>nothing here</html>")

HTTPServer(("127.0.0.1", 8899), Handler).serve_forever()
```

- Run `ExposureAgent` against `http://127.0.0.1:8899/` and confirm it reports `PASS` for both checks, even though

every request came back 200.

- Run it against a real clean site (`github.com`) and confirm the same.
- Check the full scan: `curl -X POST localhost:8000/scan -d '{"url": "github.com"}'` — four agents now in `"agents": headers, recon, tls, exposure`.

Words worth knowing

- **Soft-404** — a server that answers `200` for made-up paths too, the reason status code alone can't be trusted.
- **Content-shape check** — confirming a response actually looks like what you're checking for, not just trusting its status code.
- `re.MULTILINE` — makes `^/$` match every line in a string, not just the very start/end.

Next: A10 — DNS agent. Whether a domain's email can be spoofed — SPF and DMARC records.

A10 — DNS agent

Status: *done.* `backend/agents/dns_email.py` — `DNSAgent` checks whether a domain's email can be spoofed, by reading its SPF and DMARC DNS records.

What we built

`DNSAgent` reads two public DNS records and asks one question: **if someone forges an email claiming to be from this domain, would a receiving mail server have any way to tell?**

- **SPF** — a text record listing which mail servers are allowed to send email as this domain.
- **DMARC** — a text record saying what a receiving server should *do* when a message fails that check (reject it, send it to spam, or do nothing).

Both are ordinary public DNS lookups — the same reads any real mail server does before deciding whether to

accept a message.

The one big idea: TXT records, and reading them correctly

DNS's best-known job is turning a hostname into an IP address. A **TXT record** is different — it's a free-text field attached to a domain, and the internet has adopted it as a general place to publish small facts about a domain. SPF and DMARC are both just TXT records with an agreed-upon format.

A real domain often has many unrelated TXT records at once (Google, Stripe, and other services all ask you to "prove you own this domain" by publishing a specific string there). So the code doesn't assume the first TXT record is the right one — it searches for the one that actually starts with `v=spf1`:

```
def _find_record(records: list[str], prefix: str) -> str
| None:
    for record in records:
        if
```

```
record.strip().lower().startswith(prefix.lower()):  
    return record.strip()  
  
return None
```

An SPF record ends with an `all` rule that says what to do about senders it didn't list — `-all` (reject them), `~all` (flag them as suspicious), `?all` (no opinion, same as nothing), or `+all` (allow literally everyone, which is worse than having no SPF at all). A DMARC record has a `p=` policy — `reject`, `quarantine` (spam folder), or `none` (just watch and report, don't block anything).

Two real discoveries while testing this: `gmail.com`'s SPF record uses `redirect=_spf.google.com` instead of ending in `all` — a valid mechanism meaning "go check that other domain's record instead," which the code checks for explicitly so it isn't wrongly flagged as having no policy. And both `gmail.com` and `python.org` have DMARC set to `p=none` — even big, well-run domains often leave DMARC in "monitor only" mode rather than fully enforcing it.

The actual code

```
def _query_txt(name: str) -> list[str]:
    try:
        answer = dns.resolver.resolve(name, "TXT")
    except (dns.resolver.NXDOMAIN,
            dns.resolver.NoAnswer):
        return []
    ...
```

`dnspython`'s resolver is blocking (not async), so it's called through `asyncio.to_thread(...)` from `scan()` — the same trick used for TLS lookups in A8, so this one slow lookup doesn't freeze the whole event loop. Two exceptions both just mean "nothing here": `NXDOMAIN` (the name doesn't exist at all) and `NoAnswer` (the name exists but has no TXT record) — both happen for real, so both need catching.

```
all_token = next((t for t in tokens if
    t.lower().lstrip("+~?") == "all"), None)
qualifier = all_token[0] if all_token[0] in "+~?" else
    "+"
```

`.lstrip("+~?")` strips any of those symbols off the front of the token. A bare `all` (no symbol) has nothing to strip, so it's treated as `+all` — the permissive default, per the SPF spec.

Try it

Look up any domain's own records, no project code involved:

```
cd backend

./venv/Scripts/python.exe -c "
import dns.resolver
for name in ['example.com', '_dmarc.example.com']:
    for rdata in dns.resolver.resolve(name, 'TXT'):
        print(name, '->', b''.join(rdata.strings))
"
```

- Run `DNSAgent` against `example.com` (should be clean), `gmail.com` (SPF passes via `redirect=`, DMARC warns on `p=none`), and `badssl.com` (fails both — no records at all).

- Check the full scan: `curl -X POST localhost:8000/scan -d '{"url": "github.com"}'` — five agents now in `"agents"`.

Words worth knowing

- **TXT record** — a free-text DNS record, used to publish small facts about a domain (SPF, DMARC, and site-ownership strings all live here).
- **SPF** — lists which mail servers are allowed to send email as a domain.
- **DMARC** — says what to do with mail that fails SPF, and where to report it.
- `p=none` — DMARC in "watch only" mode; common even on major domains.

Next: A11 — Parallel. All five agents run at once via `asyncio.gather`, instead of one after another.

A11 — Parallel

Status: done. `orchestrator.py`'s `run_scan()` now runs all five agents with `asyncio.gather` instead of one at a time. Measured on a real site: sequential took ~2597ms, parallel took ~1308ms — close to the slowest single agent, not the sum of all five.

Checkpoint 2 reached.

What we built

One change to `orchestrator.py`:

```
# before: one at a time
agent_results = [await agent_cls().run(context) for
agent_cls in AGENTS]

# after: all five, together
agent_results = await asyncio.gather(
    *(agent_cls().run(context) for agent_cls in AGENTS)
)
```

Nothing about any individual agent changed — every agent already worked with `await`. Only the *order of waiting* changed: instead of "wait for agent 1, then start agent 2," it's now "start all five, then wait for whichever finishes last." The result: a full scan roughly halved in time.

The one big idea: doing things one-at-a-time vs. starting them together

Imagine making tea and toast, one person, no helpers.

One at a time: put the kettle on, wait for it to boil, pour the tea. *Then* put bread in the toaster, wait for it to pop, butter it. Total time = boiling time + toasting time.

Started together: put the kettle on. *While it's boiling*, put the bread in the toaster. Now both are happening during the same stretch of time — you're not doing two things at the exact same instant, but you're not sitting idle waiting for the kettle either. Total time is roughly whichever one takes longer, not both added up.

That's the whole idea behind `asyncio.gather`. Each agent spends most of its time *waiting* — for a web server or a DNS server to reply. Running them "one at a time" means waiting for agent 1's network reply before even starting agent 2's request. Running them together means all five requests go out immediately, and Python just waits for whichever one takes longest, while the other four wait in the background. There's still only one Python thread doing this — it's not literally doing five things at the same instant, just overlapping the waiting.

The actual code

```
async with httpx.AsyncClient(timeout=10.0) as client:
    context = ScanContext(url=url, client=client)
    agent_results = await asyncio.gather(
        *(agent_cls().run(context) for agent_cls in
          AGENTS)
    )
```

`*(... for agent_cls in AGENTS)` builds one coroutine per agent and unpacks them as separate arguments to `gather`

— the same as writing

```
asyncio.gather(HeadersAgent().run(context),  
ReconAgent().run(context), ...)
```

 by hand, just generated from the `AGENTS` list. `gather` always returns results in the order the coroutines were given, not the order they actually finished in — important, because the report's agent list needs to be stable, not shuffled by network timing.

One thing worth knowing: `gather`, by default, if *any* coroutine raises an exception, immediately throws that exception and **loses the results of every other coroutine**, even ones that already finished successfully. That's safe here only because every agent's `run()` (from A3) already catches its own exceptions and never raises — so this sharp edge never actually gets triggered in practice.

Try it

Time it yourself, sequential vs. parallel, on the same site:

```
cd backend  
./venv/Scripts/python.exe -c "
```

```
import asyncio, httpx, time

from agents.base import ScanContext

from orchestrator import AGENTS, normalize_url


async def timed(label, run_all):
    url = normalize_url('github.com')

    async with httpx.AsyncClient(timeout=10.0) as client:
        ctx = ScanContext(url=url, client=client)
        start = time.perf_counter()
        await run_all(ctx)
        print(label, f'{{(time.perf_counter() - start) *
1000:.0f}}ms')


async def sequential(ctx):
    return [await a().run(ctx) for a in AGENTS]


async def parallel(ctx):
    return await asyncio.gather(*(a().run(ctx) for a in
AGENTS))


asyncio.run(timed('sequential', sequential))
```

```
asyncio.run(timed('parallel ', parallel))  
"
```

- `sequential` 's total should track the sum of all five agents; `parallel` 's should track just the slowest one.
- Compare the `duration_ms` at the top of a real `/scan` response against the sum of each agent's own `duration_ms` inside `"agents"` — the top-level number should sit close to the *largest* single one, not the sum of all five: `curl -X POST localhost:8000/scan -d "{\"url\": \"github.com\"}"`

Words worth knowing

- **Concurrency** — overlapping the *waiting* on several things at once, without necessarily doing them at the literal same instant.
- **Parallelism** — actually doing two things at the same instant (needs multiple cores/threads); not what's happening here.
- `asyncio.gather` — starts several coroutines together and returns their results as a list, in the order they were

given, once all of them are done.

Checkpoint 2 reached: a full 5-agent scan completes in about 1.3-1.5 seconds.

Next: A12 — AI analyst. A plain-English summary from the Claude API, layered on top, that still works fine with no API key present.

A12 — AI analyst

Status: *done.* `backend/ai/analyst.py` asks an LLM (Groq's free API) to write a short, plain-English summary of a scan. If there's no API key, or the call fails for any reason, the scan still returns a complete report — the summary field is just empty.

What we built

One function, `summarize()`. It takes a scan's URL, score, grade, and findings, and asks a language model to turn them into 2-4 plain sentences a non-expert could read. It's called right after scoring, and it's the only place in the whole project that touches an LLM.

(Side note: this was originally built against Anthropic's API. Arihant asked to switch to Groq instead, since its free tier needs no credit card. The shape of the code didn't change at all — just which URL it calls.)

The one big idea: graceful degradation

A feature "degrades gracefully" when losing it doesn't break anything else — you just get a slightly plainer result. Think of a restaurant's specials board: if the person who writes it calls in sick, you don't close the restaurant, you just serve the regular menu without today's specials.

`summarize()` is built exactly like that:

```
api_key = os.environ.get("GROQ_API_KEY")
if not api_key:
    return ""
...
except Exception:
    return ""
```

No API key? Return an empty string. The call fails for any reason at all — bad key, network blip, rate limit? Also return an empty string. Every single path through this function returns *something* usable, and the rest of the report (score, grade, every finding) was already fully

computed before this line even runs. Worst case, the user loses one paragraph — nothing else.

The actual code

```
async def summarize(url, score, grade, findings) -> str:
    api_key = os.environ.get("GROQ_API_KEY")
    if not api_key:
        return ""

    try:
        async with httpx.AsyncClient(timeout=15.0) as
client:
            response = await client.post(
                _GROQ_URL,
                headers={"Authorization": f"Bearer
{api_key}"},
                json={
                    "model": _MODEL,
                    "max_completion_tokens": 800,
                    "reasoning_effort": "low", # keep it
fast and cheap
```

```

        "messages": [
            {"role": "system", "content":
_SYSTEM_PROMPT},
            {"role": "user", "content":
_build_prompt(url, score, grade, findings)},
        ],
    },
)
response.raise_for_status()
return response.json()["choices"][0]
["message"]["content"].strip()

except Exception:
    return ""

```

Calling an LLM is really just an HTTP POST — the same shape as any other API call this project makes. `messages` has a `"system"` entry (instructions for how the model should behave) and a `"user"` entry (the actual data — our findings). `response.raise_for_status()` turns a bad HTTP status (401, 429, etc.) into a Python exception, which the `except Exception` below then swallows — one line doing what would otherwise need a manual status check.

A real bug found while testing: `max_completion_tokens` was originally 200, and the summary kept coming back empty. Turns out this model "thinks" before it writes, and that thinking eats into the same token budget — 200 wasn't enough room left for the actual sentences. Raising it to 800 and setting `reasoning_effort: "low"` fixed it.

Try it

- Rename `backend/.env` temporarily, restart the server, and POST to `/scan`. Every field comes back full except `summary`, which is `""`.
- Put a real `GROQ_API_KEY` back and scan the same site again — `summary` is now a real paragraph, written from that scan's actual findings.
- Read `_SYSTEM_PROMPT` in `backend/ai/analyst.py` — it's just plain English instructions telling the model how to write, no special syntax involved.

A few words worth knowing

- **Graceful degradation** — an optional feature can fail or be missing entirely without breaking the core

product.

- `os.environ` — Python's view of environment variables; `.get()` returns `None` if a key was never set.
 - `.env` **file** — keeps secrets like API keys out of the actual code (and out of git).
 - `response.raise_for_status()` — turns a bad HTTP response into a Python exception you can catch, instead of checking the status by hand.
-

Next: A13 — Next.js up. The frontend begins.

A13 — Next.js up

Status: done. `npm run dev --prefix frontend` serves a styled input screen at `localhost:3000`, using the palette and fonts from [../DESIGN.md](#). Nothing is interactive yet — that's next.

This is the first frontend note, and you're new to both JavaScript and React, so it starts from scratch.

What we built

Three files: `frontend/app/page.tsx` (the screen you see — a heading, a sentence, a text box, a button), `frontend/app/layout.tsx` (the wrapper every page sits inside — loads the fonts, sets up `<html>` / `<body>`), and `frontend/app/globals.css` (where the design's colors and fonts become real CSS values). The button doesn't do anything yet — that's next note's job.

The one big idea: a component is just a function that returns UI

That's the whole idea React is built on. A **component** is a plain JavaScript function. It takes some data in, and it returns a description of what should appear on screen.

```
function Greeting() {  
  return <p>Good morning.</p>;  
}
```

That `<p>Good morning.</p>` sitting inside a JavaScript function is called **JSX** — it looks like HTML, but it isn't a string and isn't real HTML. It's JavaScript syntax that gets converted into plain function calls before the browser ever sees it, so you can treat it like any other value: put it in a variable, return it, pass it around.

One hard rule: a component's name must start with a capital letter (`Greeting`, not `greeting`) — that's how React tells "my component" apart from a real HTML tag like `<p>`.

Where does the URL come from? In FastAPI you wrote `@app.get("/health")` — the string is the URL. Next.js does the same job with folders instead: a file at `app/page.tsx` becomes `/`, a file at `app/report/page.tsx` would become `/report`. The folder path literally is the route. This is called the **App Router**.

Second idea: Tailwind — styling written right where it's used

Normal CSS: give something a name, then describe it somewhere else.

```
.notice { padding: 1rem; font-size: 0.875rem; }
```

```
<div class="notice">Closed Sundays.</div>
```

Tailwind skips the naming step — you list the descriptions directly as classes:

```
<div className="p-4 text-sm">Closed Sundays.</div>
```

`p-4` is padding, `text-sm` is font size. It looks noisy at first, but you never have to go hunting in a second file to find out how something is styled — it's right there on the element. (JSX uses `className` instead of `class`, because `class` is already a reserved JavaScript word.)

The actual code

```
// frontend/app/layout.tsx – the wrapper around every
page
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body className="font-body min-h-full flex flex-
col">{children}</body>
    </html>
  );
}
```

`children` is whatever page is currently being shown — the layout doesn't know or care which one, it just drops it into that spot.

```
// frontend/app/page.tsx — the actual screen at "/"
export default function Home() {
  return (
    <main className="flex flex-1 items-center justify-
center px-6 py-24">
      <h1 className="font-display text-5xl sm:text-6xl">
        Every site leaves<br />a record.
      </h1>
      <input className="border-b border-rule bg-
transparent pb-3 font-mono text-lg" />
      <button type="button" className="glass px-6 py-3
font-mono text-xs uppercase">
        Inspect
      </button>
    </main>
  );
}
```

`export default` is required on every `page.tsx` and `layout.tsx` — it's how Next.js knows which function to render. The button is `type="button"` on purpose, so it can't accidentally submit or reload the page before it's wired up.

Colors and fonts come from `globals.css` the same way:

```
@theme {  
  --color-ink: #0e0e0d;  
}
```

Declaring `--color-ink` here automatically gives you Tailwind classes like `bg-ink` and `text-ink` — no extra config file needed.

Try it

- Rename `Home` to `home` (lowercase) in `page.tsx` — the page breaks. React decides "component or HTML tag" purely from that first letter.
- Open the page and view source (Ctrl+U). The heading text is already in the raw HTML — nothing had to run in the browser to produce it.
- Change `--color-ink` in `globals.css` to a different hex and refresh — the whole background changes from one line.

A few words worth knowing

- **Component** — a function that returns UI. Must start with a capital letter.
- **JSX** — HTML-like syntax inside JavaScript; not a string, not real HTML.
- **App Router** — Next.js routing where a file's folder path is the URL.
- **Tailwind utility class** — a class that does one styling job (`p-4` , `text-sm`), written directly on the element instead of in a separate file.
- `export default` — marks the one main thing a file exports; required on every page/layout file.

Next: A14 — making the button actually do something.

A14 — It talks

Status: done. Typing a URL into `localhost:3000` and clicking *Inspect* runs a real scan against the real backend and shows the score, grade, and findings on screen. The backend gained CORS support so the browser actually allows this.

What we built

Three changes: the backend now sends CORS headers (`backend/main.py`) so the browser will let our frontend read its responses; the frontend gained a way to talk to the backend over HTTP (`frontend/lib/api.ts`); and `page.tsx` became interactive — it remembers what you typed, whether a scan is running, and what result came back.

The one big idea: state

A component is just a function, and functions forget everything the moment they return. But a page needs to

remember things — what you typed, whether a scan is running. That's what **state** is: a variable that, when you change it, tells React "redraw the screen."

```
const [count, setCount] = useState(0);  
  
// count      -> the current value  
  
// setCount() -> change it AND trigger a redraw
```

A plain variable doesn't work for this. If you wrote `let count = 0` inside the component function, every redraw would call the function again from scratch and reset it back to `0`. `useState` is what makes a value survive between redraws, and what tells React to redraw at all when it changes.

Our page uses four of these side by side:

```
const [url, setUrl] = useState("");  
const [isScanning, setIsScanning] = useState(false);  
const [report, setReport] = useState<ScanReport | null>  
(null);  
const [error, setError] = useState<string | null>(null);
```

Each one answers an independent question — what's typed, are we busy, did we get a report, did we get an error.

Second idea: CORS — the browser's "who's allowed to ask" rule

Our frontend runs on `localhost:3000`, our backend on `localhost:8000`. Different port means the browser treats them as completely different websites — this is the **same-origin policy**, and it exists so a random website you visit can't quietly read data from your bank's site just because your browser happens to be logged in there.

Without any changes, a request from the frontend to the backend would still run and get an answer — but the browser would throw the answer away and report an error, because the backend never said the frontend was allowed to read it.

CORS is the backend explicitly granting that permission:

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["http://localhost:3000",  
"http://127.0.0.1:3000"],  
    allow_methods=["GET", "POST"],  
    allow_headers=["Content-Type"],  
)
```

Only these two exact addresses are allowed — not `allow_origins=["*"]`, which would let any website on the internet trigger a scan through a visitor's browser. Worth knowing: CORS only restrains *browsers*. A tool like `curl` ignores it completely and can call the API directly either way — CORS isn't API security, it's a browser-only politeness rule.

The actual code

Calling the backend uses `fetch` — the browser's built-in HTTP client:

```
export async function downloadReportPdf(report:
ScanReport): Promise<void> {
  const response = await fetch(`${API_BASE}/scan/pdf`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(report),
  });

  if (!response.ok) {
    throw new Error(`PDF export failed
(${response.status})`);
  }

  // ...save the response as a file
}
```

One surprising thing about `fetch`: it does **not** raise an error for a 404 or 500 response — only for a request that never completed at all (no connection, DNS failure). A "bad" response still counts as success as far as `fetch` is concerned, which is why `!response.ok` has to be checked by hand.

The URL input is a **controlled input** — React state, not the browser, owns what's shown in the box:

```
<input
  value={url}
  onChange={(event) => setUrl(event.target.value)}
/>
```

Every keystroke calls `setUrl`, which updates state, which redraws the box showing the new text. Submitting the form calls `event.preventDefault()` first — without it, the browser's decades-old default behavior kicks in: reload the whole page and throw away all your state.

Try it

- Comment out the `add_middleware(...)` block in `main.py`, restart, and try a scan from the browser. It fails — but the exact same request via `curl` still works fine. That's the browser-only part of CORS, proven.
- Delete `onChange` from the `<input>`. Typing stops doing anything, because nothing updates state anymore.

- Remove `event.preventDefault()` from the submit handler. Submitting now reloads the whole page instead of running the scan.

A few words worth knowing

- **State** — a variable that redraws the UI when it changes; created with `useState`.
- **Controlled input** — an input whose value comes from state, not from the browser's own memory of what you typed.
- `fetch` — the browser's built-in way to make HTTP requests. Doesn't throw on a bad status code, only on a failed connection.
- **Origin** — scheme + host + port. All three must match to count as "the same site."
- **CORS** — headers a server sends to tell the browser which other origins are allowed to read its responses.

Next: A15 — the real report screen.

A15 — Report view

Status: done. Scanning a site now shows a real report page: a score ring, an AI summary, findings grouped by category, and a log of what each agent did.

What we built

Four new files under `frontend/components/`: `ScoreRing.tsx` (draws the grade as a circle), `FindingRow.tsx` (one finding, one block of text), `AgentLog.tsx` (the five agents and how long each took), and `Report.tsx` (puts the other three together into the full page). Once a scan finishes, `page.tsx` swaps its placeholder for `<Report report={report} />`.

The one big idea: props are just parameters

A React component is a function. **Props are how you pass it arguments.**

```
function Greeting({ name }) {  
  return <p>Hello, {name}</p>;  
}  
  
<Greeting name="Priya" />
```

`name="Priya"` at the call site and `{ name }` in the function signature are the whole mechanism — React bundles every attribute you write on a tag into one object and hands it to the function as its single argument. This is exactly Python's keyword arguments:

```
def greeting(name):  
    return f"Hello, {name}."  
  
greeting(name="Priya")
```

The one rule worth remembering: **props flow one way, parent to child**. A component can read its props but can't change them — `ScoreRing` gets `score` and `grade` from `Report`, but has no way to hand a different number back up.

```
export function ScoreRing({ score, grade }: { score:
number; grade: string }) {
```

The second `{ }` is just the TypeScript type of the props object — "this component takes exactly these two things." Mix them up at the call site and it's a compile error, not a bug you find by staring at a broken page.

The actual code

`Report.tsx` doesn't know how to draw a circle or format a finding — it just imports the three smaller components and arranges them:

```
<ScoreRing score={report.score} grade={report.grade} />
...
{group.problems.map((finding) => (
  <FindingRow key={finding.id} finding={finding} />
))}
...
<AgentLog agents={report.agents} totalMs=
{report.duration_ms} />
```

This is called **composition** — building one big screen from several small, single-purpose pieces. Same idea as `orchestrator.py` combining five agents without knowing how any single one works internally.

A couple of smaller things worth a glance in the source:

- `Report.tsx` computes `groups` and `problemCount` fresh on every render instead of storing them in `useState` — they're fully determined by `report.findings`, which is already a prop, so recalculating avoids two copies of the same fact drifting apart.
- `ScoreRing` draws its arc with `strokeDasharray / strokeDashoffset`: the circle's outline is one dash the length of the whole circumference, and the offset hides the part that shouldn't show. At `score=100` nothing is hidden; at `score=0` the whole dash is hidden. The ring never changes colour with the score — `DESIGN.md` reserves the accent red for Critical findings only, so the arc's *length* carries the information instead of a colour.
- Findings only show a description, evidence box, or remediation line if that field is actually non-empty

`( {finding.evidence && (...)} )` — every one of those fields is optional in the backend model.

Try it

1. Scan two very different sites (e.g. `github.com` and `neverssl.com`) and watch the ring's arc length change while its colour stays the same.
2. In `ScoreRing.tsx`, hardcode `strokeDashoffset={0}`. Every scan now shows a full ring, proving the offset — not some hidden per-grade logic — draws it.
3. Remove `key={finding.id}` from the `.map()` in `Report.tsx`, run a scan, and check the browser console — React warns about a missing `key` on list items.
4. In `FindingRow`, try `finding.severity = "Critical"` at the top of the function body. Nothing breaks, but `Report.tsx` never sees the change — props are a snapshot handed down, not a live link back up.

A few words worth knowing

- **Props** — the object a component receives as its one argument, built from the attributes at its call site. Flow one direction: parent to child.
 - **Composition** — building a screen from small, focused components instead of one giant one.
 - **Derived data** — a value computed fresh from props/state every render, instead of stored separately, so it can never fall out of sync.
 - `strokeDasharray` / `strokeDashoffset` — SVG's way of drawing a partial circle: the dash pattern length, and how far into it drawing starts.
-

Next: A16 — live progress, watching each agent finish in real time instead of waiting for all five at once.

A16 — Live progress and Server-Sent Events

***Status:** done. The waiting screen now lights up each agent the instant it actually finishes, instead of five names pulsing together and then all appearing at once.*

What we built

A second way to run a scan, alongside the original `POST /scan`. `backend/orchestrator.py` gained `run_scan_stream()`, which reports each agent's result the moment that agent is done. `backend/main.py` gained `GET /scan/stream`, which turns that into a live stream the browser can read.

`frontend/lib/api.ts` gained `streamScan()`, which opens that stream and updates the page as each result arrives.

`POST /scan` still exists unchanged — this is a second option, not a replacement.

The one big idea: `asyncio.gather` VS

`asyncio.as_completed`

A11 already made the five agents run *concurrently* with `asyncio.gather` — it starts all five at once, but only hands back results after **all** of them are done, in the order you listed them.

`asyncio.as_completed` runs the exact same coroutines, concurrently, the same way — but hands each one back **the moment it finishes**, in real finishing order:

```
import asyncio

async def task(name, delay):
    await asyncio.sleep(delay)
    return name

async def main():
    coros = [task("slow", 2), task("fast", 0.3),
task("medium", 1)]
    for coro in asyncio.as_completed(coros):
        print(await coro)    # fast, medium, slow -- real
```

```
finishing order
```

```
asyncio.run(main())
```

Nothing about *how fast* the work happens changes — same concurrency, same total time. The only difference is whether you find out about each result as it's ready, or all together at the end. That's why `run_scan_stream` uses it:

```
coros = [agent_cls().run(context) for agent_cls in
AGENTS]

for coro in asyncio.as_completed(coros):
    result = await coro
    agent_results.append(result)
    yield ("agent", result)
```

A function with both `yield` and `await` is called an **async generator** — the consumer loops over it with `async for` instead of plain `for`.

The other new idea: Server-Sent Events

Ordinary HTTP is "client asks once, server answers once."
Server-Sent Events (SSE) get around that: the server sends one HTTP response as usual, but never closes it — it keeps writing more text into the same open connection, and the browser reads that as a sequence of separate messages.

```
event: agent
data: {"agent": "dns", "duration_ms": 155}

event: done
data: {"score": 54, "grade": "F"}
```

Each message is an `event:` name plus a `data:` payload (just text — SSE doesn't parse JSON for you), and a blank line ends each message.

FastAPI's `StreamingResponse` builds a response from a generator instead of one finished value, sending each piece of text as it's produced:

```
async def events():
    async for event_name, payload in
run_scan_stream(url):
    yield _sse(event_name, payload.model_dump_json())

return StreamingResponse(events(),
media_type="text/event-stream")
```

One consequence: once the first byte of a streaming response goes out, the status code (`200`) is locked in. So a bad URL can't become a clean `400` the way it does for `POST /scan` — instead it shows up as an `event: failed` message inside an otherwise- `200` stream.

On the browser, `EventSource` is the built-in client for reading an SSE stream, no library needed. Two things about it shaped the code around it: it can only send `GET` (no body — hence `?url=...` as a query parameter instead of JSON), and it **auto-reconnects** whenever the connection closes, even a normal successful end. Left alone, a finished scan would silently trigger a second scan

a few seconds later — the fix is calling `source.close()` once the `"done"` event arrives.

`streamScan()` can't just `return` one value the way `A14's runScan` did — a scan produces six separate results over time (five agents plus a report), and a `Promise` only ever resolves once. Instead it takes callback functions and calls the right one as each result shows up:

```
streamScan(url, {
  onAgent: (result) => setAgentResults((prev) => ({
    ...prev, [result.agent]: result })),
  onDone: (report) => setReport(report),
});
```

`{ ...prev, [result.agent]: result }` copies everything already known into a *new* object, then adds the latest agent's result — it has to be new because React checks whether state changed identity, not its contents; mutating `prev` in place would look like nothing happened.

Try it

1. Run this note's `as_completed` example with delays in a "wrong" order and confirm `gather` prints results in declared order while `as_completed` prints them in actual finishing order.
2. Remove the `source.close()` call after `"done"` and watch the Network tab a few seconds after a scan finishes — a second request fires on its own.
3. Run `curl -D- -o/dev/null "localhost:8000/scan/stream?url=bad-scheme://x"` and confirm the status is still `200` — the real error is inside the stream.
4. Watch the five agent names in `page.tsx` light up one at a time, in a different order on different scans, matching real completion time.

A few words worth knowing

- `asyncio.as_completed` — runs coroutines concurrently, same as `gather`, but yields each result as it finishes rather than all together at the end.
- **Async generator** — a function using both `async / await` and `yield`.

- **Server-Sent Events (SSE)** — a one-way stream of text messages over one long-lived HTTP response.
 - `StreamingResponse` — a response built from a generator instead of one finished value.
 - `EventSource` — the browser's built-in SSE client. GET-only, no body, auto-reconnects unless you call `.close()`.
-

Next: A17 — PDF export, using a real headless browser to print the report.

A17 — PDF export and Playwright

***Status:** done. A "Download PDF" button next to the score ring saves the report you're looking at as a real PDF file.*

What we built

The button POSTs the report already showing on screen to a new backend endpoint, `POST /scan/pdf`. That endpoint rebuilds the report as a plain HTML page — by hand, in Python, since there's no React on the server — hands that HTML to a real but invisible ("headless") browser via a library called Playwright, asks it to print, and streams the resulting PDF bytes back. The frontend turns those bytes into a file the browser saves to disk.

The one big idea: a headless browser

"Headless" means a real browser engine — the same code that runs Chrome — running with no visible window, driven entirely by code: open a page, set some HTML, ask it to do something, close it.

Why reach for a whole browser just to make a PDF? A PDF is basically "print this webpage," and browsers have done that correctly for decades — real CSS, real fonts, real layout, for free. An HTML-to-PDF *library* has to reimplement all of that itself, and usually has gaps (our score ring uses inline SVG, which plenty of lightweight PDF libraries choke on). Using an actual browser sidesteps the problem instead of solving it twice.

Playwright is the library that drives one. Small standalone example — run `python -m playwright install chromium` once, then run this:

```
import asyncio

from playwright.async_api import async_playwright


async def main():
    async with async_playwright() as p:
```

```
browser = await p.chromium.launch()
page = await browser.new_page()
await page.set_content("<h1 style='color:
teal'>Hello, PDF</h1>")
await page.pdf(path="hello.pdf")
await browser.close()

asyncio.run(main())
```

A real `hello.pdf` appears, with proper font rendering, and no window ever opened. `backend/report/pdf.py`'s `generate_pdf()` does exactly this, just with a much longer HTML string, and `page.pdf()` returning bytes instead of writing to disk, since those bytes need to travel over HTTP.

The other thing worth knowing: escaping untrusted text

`render_html()` builds the page from Python f-strings — including text that came from a *scanned website*, like a finding's `evidence` field, which might hold the literal text of an HTTP header. If that text contains `<` and `>` and gets

dropped into the HTML unescaped, a stray `<script>` tag would genuinely execute inside the page being printed, since Playwright is a real browser.

The fix is `html.escape()`, applied to every piece of dynamic text:

```
from html import escape

comment = "nice site! <script>alert('hi')</script>"
print(f"<p>{escape(comment)}</p>")

# <p>nice site!
<script>alert('hi')</script></p>
```

It still reads the same to a person, but a browser now treats it as inert text instead of running it as markup. Every `escape(...)` call in `pdf.py` exists for this reason.

The actual code

`render_html()` is the Python twin of `Report.tsx` — same score-ring math, same category grouping — rebuilt by hand because Python can't run a React component. The

endpoint takes the whole report as its request body, not just a URL:

```
@app.post("/scan/pdf")
async def scan_pdf(report: ScanReport) -> Response:
    pdf_bytes = await generate_pdf(report)
    ...
    return Response(content=pdf_bytes,
media_type="application/pdf", ...)
```

Re-running the scan server-side was rejected on purpose — a site's headers could change between the on-screen scan and the PDF export moments later, so the file could disagree with what's on screen. Sending the whole report guarantees the PDF matches exactly what the user is looking at.

On the frontend, a PDF is binary data, not JSON —

`fetch()` hands that back as a `Blob`, and

`URL.createObjectURL()` turns it into a temporary link an

`<a>` tag can "download":

```
const blob = await response.blob();
const url = URL.createObjectURL(blob);
const link = document.createElement("a");
link.href = url;
link.download = "sentinels-example-com.pdf";
link.click(); // never added to the page -- click()
              alone starts the save
```

One real bug worth knowing: on Windows, running the server with `uvicorn --reload` switches Python to an event loop that can't start subprocesses — and Playwright's browser is a subprocess. `pdf.py` works around this by running the PDF generation on its own dedicated event loop in a separate thread, so exporting still works even under `--reload`.

Try it

1. Time `generate_pdf()` and compare to a scan's own `duration_ms` — launching a whole browser process costs more than all five scan agents combined.

2. POST a hand-crafted report with a finding title of `<b>bold text</b>` to `/scan/pdf` and open the PDF — the title should read as literal text, not render bold. Comment out that one `escape()` call and try again to see it actually render bold.
3. In `generate_pdf()`, change `print_background=True` to `False` and export again — the dark page background disappears, since browsers skip background colours when printing unless told otherwise.

A few words worth knowing

- **Headless browser** — a real browser engine running with no visible window, driven entirely by code.
- **Playwright** — the library used here to launch and control headless Chromium.
- `html.escape()` — converts `<`, `>`, `&`, and quotes into inert HTML-entity text, so untrusted text can't be interpreted as markup.
- **Blob** / **object URL** — the browser's way of holding raw binary data and giving it a temporary URL an `<a download>` can point at.

Next: the roadmap's remaining follow-ups — see

`docs/ROADMAP.md` .

A17b — Scroll drift and refs

Status: done. `frontend/lib/useScrollDrift.ts` makes the Agent Log panel drift slightly slower than the page as you scroll.

What we built

The Agent Log panel on the report screen now moves a little slower than the rest of the page when you scroll — it "hangs back" instead of scrolling in lockstep with everything around it. It's built as a reusable hook, so any other panel can get its own drift speed later in a single line.

The one big idea: a custom hook

React ships some hooks for you — `useState` is one you've already used. A **custom hook** is just a function you write yourself that also starts with `use` and is allowed to call those built-in hooks inside it. That's the whole rule.

Without it, every panel that wants to drift would need the same few lines copy-pasted into it — instead, `AgentLog.tsx` writes one line, `useScrollDrift(0.06)`, and never has to know how the drifting works.

```
function useCounter(start: number) {  
  const [count, setCount] = useState(start);  
  return { count, increment: () => setCount((c) => c + 1)  
};  
}
```

Any component can call `useCounter(0)` and get its own independent counter — same idea as `useScrollDrift` giving each panel its own independent listener.

Reaching into the real page: `useRef` + `useEffect`

Normally in React you don't touch the actual page elements — you change state, and React updates the screen for you. But to make a panel drift, we need to grab the *real* HTML element and move it directly. Two tools do that:

- `useRef` gives you a box, `{ current: ... }`, that React fills in with the real DOM element once it exists. `<div ref={someRef}>` is how you say "hand me this one."
- `useEffect` runs code *after* React has drawn the screen — the right moment to attach something like a scroll listener, since the element needs to already exist. It can also return a cleanup function that runs when the component goes away, so old listeners don't pile up.

Think of `useRef` as a sticky note with an address on it, `useEffect` as "once the building exists, go do something at that address."

The actual code

```
export function useScrollDrift<T extends HTMLElement>
(speed: number) {
  const ref = useRef<T | null>(null);

  useEffect(() => {
    const el = ref.current;
    if (!el) return;
```

```
// Respect the OS "reduce motion" setting – just
don't animate at all.

if (window.matchMedia("(prefers-reduced-motion:
reduce)").matches) {
    return;
}

function onScroll() {
    el!.style.transform = `translateY(${window.scrollY
* speed}px)`;
}

window.addEventListener("scroll", onScroll, {
passive: true });

return () => window.removeEventListener("scroll",
onScroll);
}, [speed]);

return ref;
}
```

- `speed` is a multiplier: `0.06` means the panel moves at 6% of however far you've scrolled — subtle, not seasick.
- `el.style.transform` is set directly on the real element, not through `setState` — scroll fires very often, so re-rendering React each time would be wasteful.
- `{ passive: true }` promises the browser this listener won't block scrolling, so scrolling can start immediately.

In `AgentLog.tsx`: `const driftRef = useScrollDrift<HTMLDivElement>(0.06);`, then `<div ref={driftRef} className="glass ...">`.

Try it

- In `AgentLog.tsx`, change `0.06` to `0.3` and scroll — much more noticeable drift. Try `0` — the panel stops moving relative to the page entirely.
- Turn on "reduce motion" in your OS accessibility settings, reload the page, and scroll — the panel should stay still.
- Comment out the `return () => window.removeEventListener(...)` line, then navigate to

the report and back a few times — scrolling gets janky as old listeners pile up.

Words worth knowing

- **Custom hook** — a function you write, named `use...`, that calls real hooks inside it.
 - `useRef` — a box that survives re-renders and can hold a real DOM element.
 - `useEffect` — runs code after React updates the screen; can return a cleanup function.
 - **Passive listener** — a scroll/touch listener that promises not to block the browser's default handling.
-

Next: 17c — giving each panel its own drift speed, and capping it so it doesn't overlap the next section.

A17c — Drift speeds and clamping

***Status:** done. The header and each findings category now drift at their own speed, and `useScrollDrift` can cap how far that drift goes.*

What we built

Only the Agent Log panel drifted before. Now the report's header and each findings category ("Headers", "TLS", "DNS", ...) drift too, each at a different speed — different rates moving together is what reads as depth, instead of one flat plane. Getting there surfaced a real bug: on a long report, an uncapped drift on a findings category made its text overlap the "Agent log" heading below it. The fix was teaching `useScrollDrift` an optional cap.

The one big idea: you can't call a hook inside a loop

`Report.tsx` renders one section per findings category using `groups.map(...)`. The obvious move is to call `useScrollDrift` once per category inside that loop — but React doesn't allow it. Hooks must run the same number of times, in the same order, on every render, and a `.map()` over a list that can grow or shrink (a report can have 3 categories or 5) breaks that promise.

The fix: pull the thing that needs its own hook call into its own small component — `groups.map(group => <FindingsCategory group={group} />)`. Each `<FindingsCategory>` calls `useScrollDrift` once, which is fine: from React's point of view that's five separate components each independently using a hook, not one component looping.

```
function Counter({ label }: { label: string }) {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>
    {label}: {count}</button>;
}

// items.map(item => <Counter key={item} label={item} />)
- fine, each
```

```
// instance calls useState once. useState directly inside  
the .map()  
  
// callback instead would NOT be fine.
```

Clamping: capping a value that would otherwise grow forever

The drift was just `scrollY * speed` — no limit. Fine on a short page, but findings categories sit at different depths depending on how many findings a scan turns up, and a category drifting at speed `0.08` could move ~200px by the bottom of a long report — more than the fixed 80px gap to the next heading, so the text visibly overlapped it.

The fix is `Math.min(offset, cap)`: once the drift would go past the cap, just use the cap instead.

```
function healthBarWidth(currentHp: number, maxHp:  
number): number {  
  const rawPercent = (currentHp / maxHp) * 100;  
  return Math.min(rawPercent, 100); // healing past max
```

```
still shows a full bar  
}
```

The actual code

`useScrollDrift` gained an optional second parameter:

```
export function useScrollDrift<T extends HTMLElement>(  
  speed: number,  
  maxOffsetPx?: number,  
) {  
  // ...  
  function onScroll() {  
    const offset = window.scrollY * speed;  
    const clamped =  
      maxOffsetPx === undefined ? offset :  
      Math.min(offset, maxOffsetPx);  
    el!.style.transform = `translateY(${clamped}px)`;  
  }  
  // ...  
}
```

If `maxOffsetPx` isn't passed, nothing changes — `AgentLog.tsx`'s existing `useScrollDrift(0.06)` call still drifts uncapped, exactly as before. The two new calls in `Report.tsx` both pass a cap:

```
const headerDriftRef = useScrollDrift<HTMLElement>(0.03,
32);
// and, once per category:
const driftRef = useScrollDrift<HTMLElement>(0.08, 48);
```

The cap numbers (32px, 48px) came from actually measuring the gap between elements on a real report, not guesswork — leaving some of that gap always intact no matter how long a report gets.

Try it

- In `FindingsCategory`, drop the cap (`useScrollDrift<HTMLElement>(0.08)`), scan a site with several findings, and scroll to the bottom — watch the last category's text run into "Agent log." Put the `48` back afterward.

- Try moving `useScrollDrift(0.08, 48)` directly inside the `groups.map(...)` callback instead of inside `FindingsCategory` — React throws a dev-mode error about hooks being called conditionally.
- Log `{ offset, clamped }` inside `onScroll` and scroll a long report — the numbers match early on, then diverge once `offset` passes the cap.

Words worth knowing

- **Rules of hooks** — hooks must run the same number of times, in the same order, on every render of a component.
 - **Component extraction** — pulling repeated hook + JSX logic into its own component so each repetition is its own instance.
 - **Clamping** — bounding a value with `Math.min` (cap a max) or `Math.max` (floor a min).
-

Next: 17d — styling findings evidence as a bordered "glass" box.

A17d — Findings evidence as artifacts

Status: *done. Each finding's evidence now sits inside a bordered "glass" box instead of plain text.*

What we built

Every finding's `evidence` field — the actual header value, cert detail, or `robots.txt` line the scan read from the real site — used to look like just another paragraph of text. Now it sits inside its own bordered, frosted box with an "Evidence" label, so it visually reads as something *lifted from the scan*, not written for the report.

No new concepts here

This one's genuinely simple: everything used already exists elsewhere in the codebase.

- `.glass` — the frosted-box style (background blur + hairline border) defined in `frontend/app/globals.css`,

already used by the Agent Log panel and the "Download PDF" button.

- `{finding.evidence && (...)}` — conditional rendering, already the pattern every other optional field on `FindingRow` uses.

This step is composition, not new mechanics — taking a box style and a rendering pattern that already exist and combining them somewhere new.

The actual code

`frontend/components/FindingRow.tsx`, before:

```
{finding.evidence && (  
  <p className="mt-3 font-mono text-xs leading-relaxed  
break-words text-muted">  
    {finding.evidence}  
  </p>  
)}
```

After:

```
{finding.evidence && (  
  <div className="glass mt-3 px-4 py-3">  
    <p className="font-mono text-[9px] uppercase  
tracking-[0.25em] text-muted">  
      Evidence  
    </p>  
    <p className="mt-1.5 font-mono text-xs leading-  
relaxed break-words">  
      {finding.evidence}  
    </p>  
  </div>  
)}
```

Three small changes: wrapping it in a `.glass` box gives it a visible boundary; the "Evidence" label (small, heavily spaced-out text) names what's inside before you read it; and the evidence text itself dropped `text-muted`, since the box now does the job of marking it as secondary — the text inside reads at normal contrast.

Try it

- Delete the `glass` class from the wrapping `<div>` and reload a real scan — the evidence falls back to plain text, no visible boundary.
- Merge the label and content back into one `<p>` (`Evidence: {finding.evidence}`) and compare — notice how much harder it is to tell label from data at a glance once they're the same size and weight.
- Scan `wordpress.org` — the Recon category's "Generator meta tag" finding has real evidence lifted from the live homepage, rendered safely as plain text rather than executed as markup.

Words worth knowing

Nothing new this note — see `learning/17b-scroll-drift-and-refs.md` for the `.glass` utility and `learning/15-components-props-and-the-report.md` for conditional rendering.

Next: 17e — the report's waiting/loading state as its own set piece.

A17e — Screen 2: the set piece

Status: *done.* `frontend/components/ScanProgress.tsx` shows five glass panels, one per agent, that "materialize" the instant that agent's real result arrives.

What we built

The waiting screen used to be five plain words fading from dim to bright. Now it's five glass panels — one per scanning agent — sitting dim and "Waiting..." until that agent's real result comes in over the network. The moment it does, that one panel (and only that one) plays a short pop-in animation and shows its real finding count and timing. Nothing is faked: the order panels light up in is just whichever agent the real scan actually finished first.

The one big idea: forcing React to replay an animation

A CSS `animation` set on an element only plays when that element is first put on the page. If React just *updates* an existing element's text, the animation does **not** replay — same element, so no fresh entrance.

React decides "same element or new element?" using each item's `key` prop. Normally you pick a key that stays stable (like a database id) so React reuses the element. This component does the opposite on purpose:

```
key={result ? `${name}-done` : `${name}-waiting`}
```

Before a result arrives, the key is `"headers-waiting"`. The instant `agentResults.headers` gets set, the key becomes `"headers-done"` — a different string. React has never seen that key before, so it throws away the old panel and mounts a brand new one. That fresh mount is what makes the pop-in animation fire, exactly once, exactly when real data shows up.

Everyday version: it's like renaming a package the moment it's delivered. The delivery service sees a "new"

label and re-does the drop-off ritual, instead of just updating a sticky note on the same box.

The other concept: `prefers-reduced-motion`

Some people set an OS-level setting asking apps to cut down on motion (motion sickness, vestibular issues, or just preference). CSS can check for that setting directly, no JavaScript needed:

```
@keyframes materialize {  
  from { opacity: 0; transform: translateY(10px)  
scale(0.97); }  
  to   { opacity: 1; transform: translateY(0) scale(1); }  
}  
  
@utility materialize-in {  
  animation: materialize 550ms cubic-bezier(0.16, 1, 0.3,  
1) both;  
  
  @media (prefers-reduced-motion: reduce) {  
    animation: none;  
  }  
}
```

```
}  
  
}
```

`@keyframes` just names a sequence of styles over time ("start like this, end like this"), and `animation` is what actually plays it on an element. The `@media (prefers-reduced-motion: reduce)` block overrides that to `animation: none` for anyone who's asked their OS for less motion — checked for real by reading the built stylesheet, not just eyeballing the source.

The actual code

```
{AGENT_NAMES.map((name) => {  
  const result = agentResults[name];  
  return (  
    <div  
      key={result ? `${name}-done` : `${name}-waiting`}  
      className={`glass materialize-in px-4 py-4 ${result  
? "" : "opacity-40"}}`  
    >  
      <p className={`font-mono text-[10px] uppercase
```

```

tracking-[0.2em] ${result ? "" : "animate-pulse"}`>
    {name}
</p>
{result ? (
    result.error ? (
        <p className="mt-2 font-mono text-[10px]
leading-snug break-words">{result.error}</p>
    ) : (
        <p className="mt-2 font-mono text-[10px] text-
muted">
            {result.findings.length} checks .
{result.duration_ms}ms
        </p>
    )
) : (
    <p className="mt-2 font-mono text-[10px] text-
muted">Waiting...</p>
)}
</div>
);
}}}
```

- Waiting panels are dimmed (`opacity-40`) and pulse; done panels go full brightness — contrast by weight, not a new color.
- The error branch mirrors `AgentLog.tsx` : if an agent failed, its own error text shows instead of a check count — A3's rule that agents never crash the scan, just report their own failure.
- `page.tsx` shrank to one line — this whole block used to live there inline.

Try it

- Change the `key` to just `key={name}` (drop the `-waiting / -done` suffix), run a real scan, and watch the pop-in stop firing — panels still switch from dim to lit, but React now just updates the old element.
- In dev tools, inspect a "done" panel and confirm `animationName` shows `materialize` in its computed styles.
- Scan a domain your network can't reach and watch a panel show its own error text instead of a check count.

Words worth knowing

- **Mount-triggered animation** — a CSS animation that only plays when its element is newly inserted into the page, not when its content updates.
 - **Keying to force a remount** — deliberately changing an element's `key` so React treats it as new instead of reusing the old one.
 - `prefers-reduced-motion` — an OS-level accessibility setting; CSS can check it directly with `@media` and turn animations off.
-

Next: [17f](#) — a real bug found while testing setup instructions, and the two Windows event loops behind it.

17f — Two event loops on Windows, and the `--reload` trap

***Status:** done. `backend/report/pdf.py` now works under `uvicorn main:app --reload` on Windows. This note is about a real bug that was found and fixed, not a hypothetical.*

What we built

The README says to start the backend with `uvicorn main:app --reload`. Following that literally on a fresh clone, everything worked — scanning, scoring, the AI summary — except **PDF export**, which failed with a blank `500 Internal Server Error`. The cause had nothing to do with our PDF code: on Windows, `--reload` quietly switches the whole process to a kind of event loop that can't start another program. Playwright's entire job is to start another program (a browser). Fixed in `backend/report/pdf.py` by giving the PDF work its own, correct kind of loop, on its own thread.

The one big idea: an event loop is a specific thing, and Windows has two

You've already met `async` / `await` as "the event loop juggles many waiting tasks at once" — like tending a toaster and a pot of water at the same time instead of standing and staring at one. That's still true. What's new here: "the event loop" isn't one single thing. It's an actual object, and Python ships more than one version of it for Windows:

Loop	Can run coroutines?	Can start another program?
Proactor (Windows default)	yes	yes
Selector (the other one)	yes	no

Both loops can `await` things just fine. Only Proactor can start a **subprocess** — another program your program launches and talks to. Playwright needs exactly that: a real Chromium browser, running as its own separate process, not a Python library.

Here's the trap: uvicorn's own code

(`uvicorn/loops/asyncio.py`) does this when `--reload` is on, on Windows:

```
if sys.platform == "win32" and use_subprocess:

    asyncio.set_event_loop_policy(asyncio.WindowsSelectorEventLoopPolicy())
```

`--reload` needs to manage its own worker processes, so it deliberately switches Windows to the Selector loop — the one that *can't* start subprocesses. That's a reasonable choice for uvicorn's file-watching, but it quietly takes away an ability our PDF code depends on. Try to start a subprocess on that loop and you get a bare `NotImplementedError`, no message at all — which is exactly what showed up as our blank 500.

The fix: build the right loop by hand, on a spare thread

A loop belongs to whichever thread runs it, and our server's main thread is already busy running uvicorn's

(Selector) loop. So the fix moves the PDF work to a separate thread, and builds a Proactor loop directly on it:

```
def _render_pdf_on_own_loop(html: str) -> bytes:
    loop = asyncio.ProactorEventLoop()      # built
    directly, not asked for
    try:
        return loop.run_until_complete(_render_pdf(html))
    finally:
        loop.close()

async def generate_pdf(report: ScanReport) -> bytes:
    html = render_html(report)

    if sys.platform != "win32":
        return await _render_pdf(html)

    return await
    asyncio.to_thread(_render_pdf_on_own_loop, html)
```

- `asyncio.ProactorEventLoop()` **directly**, not `asyncio.new_event_loop()`. The normal way of asking for a loop goes through the *policy* — a process-wide setting for "what kind of loop to hand out" — and the policy is exactly what uvicorn overrode. Asking it would just hand back another broken Selector loop. Building the class directly skips that.
- `asyncio.to_thread` hands the work to a spare thread that's free to run its own loop, and `await`s the result without freezing the server. Same tool the TLS agent (A8) already uses to keep a blocking handshake off the main loop.
- **The** `sys.platform != "win32"` **check** — this whole Selector-vs-Proactor split is Windows-only, so Linux/macOS just take the normal path.

Try it

- On the fresh clone, start the backend as `uvicorn main:app` (no `--reload`) and download a PDF — it works, because the default loop never got swapped out.

- Start it again with `--reload`, before the fix — blank 500 in the browser, `NotImplementedError` in the server's terminal.
- With the fix in place, hit `/health` repeatedly while a PDF is generating. Measured: it answered in 2-3ms each time, proving the PDF work is really on a separate thread, not blocking the server.

Words worth knowing

- **Event loop** — the object that actually runs coroutines. More than one implementation exists.
 - **Subprocess** — another program your program starts and talks to.
 - **Event loop policy** — a global setting deciding which loop kind gets built; anyone in the process can change it.
 - `asyncio.to_thread` — run a blocking function on a spare thread and `await` the result, keeping the main event loop free.
-

Next: [18](#) — how this bug was actually found: testing the setup instructions themselves, on a genuinely fresh clone.

18 — Shipping: docs as a contract, and clean-room verification

***Status:** done. Roadmap complete, 18/18. A real README exists, and the setup instructions in it have actually been proven to work.*

What we built

Achievement 18 was "hand the repo to a stranger and have them run it." That meant: a real root `README.md` (what Sentinels is, the passive-only rule, two real screenshots, setup steps), a documented `backend/.env.example`, a proper `frontend/README.md`, and — the part that actually mattered — a genuine clean-room test: cloning the repo fresh, onto a machine with none of this project's history, and literally typing only the documented commands. That test found a real bug (the event-loop trap in [17f](#)), which got fixed and then re-verified the same way.

The one big idea: you can't trust your own machine to check your own instructions

Every other learning note tested whether the *code* works. This one is about testing whether the *instructions* work — and that's a different kind of test, because of one specific trap:

Your own machine already has everything installed. If the README forgets a step, your machine won't notice, because it already has that thing from six months of working on this project (or some other project entirely). The gap is invisible from exactly the computer you're writing the instructions on.

Think of it like writing a recipe from memory for a dish you've made fifty times. You know, without needing to be told, that the pan needs greasing — your hands just do it. So you never write "step 0: grease the pan." Someone following your recipe exactly, for the first time, ends up with dinner fused to the pan. The recipe isn't wrong about anything it says — it's missing something it assumed.

You can't fix this by rereading the recipe more carefully; you already believe it's complete. The only fix is handing it to someone (or something) that has never greased that pan — a genuinely fresh kitchen. For this project, that meant an actual `git clone` into a brand-new folder that had never had `.venv` or `node_modules` in it, and running only the exact commands the README says to run.

That's exactly how the `--reload` PDF bug in 17f was found. Every prior achievement's testing happened on a machine that already had Chromium installed and had never combined `--reload` with Playwright in that specific way — so nothing before this ever had a reason to hit it. The clean-room run did, on the first try.

A quick way to check your own setup docs

Everything a clean-room clone is missing is, by definition, everything `.gitignore` lists. So a useful check is: read `.gitignore` top to bottom and confirm each line has a matching command in the README that rebuilds it.

```
.venv/          <- python -m venv .venv && pip install -r
requirements.txt
node_modules/   <- npm install
.env            <- copy from .env.example, never
committed
```

If something's gitignored but has no rebuild instruction anywhere, that's the exact kind of gap clean-room testing exists to catch.

Try it

- Pick any project you've had set up a while. Write its setup steps from memory, without opening the README. Then compare against its actual `.gitignore` — something will be missing. That's the point, not a personal failing.
- For this project, check that each `.gitignore` line above really does have a matching command in the root `README.md`.
- Read [17f](#) for the concrete bug this exact process caught.

Words worth knowing

- **Clean-room verification** — testing written instructions by following them literally, on a machine that's never benefited from any undocumented setup already sitting on your own.
- **The invisible-step trap** — a setup step you do automatically, without writing it down, because your own machine already satisfies it.
- **`.gitignore` as a checklist** — every gitignored path is something a fresh clone won't have; each one should map to a written command that rebuilds it.

Next: nothing queued — the roadmap is complete (18/18).